



School of Artificial Intelligence
Jilin University

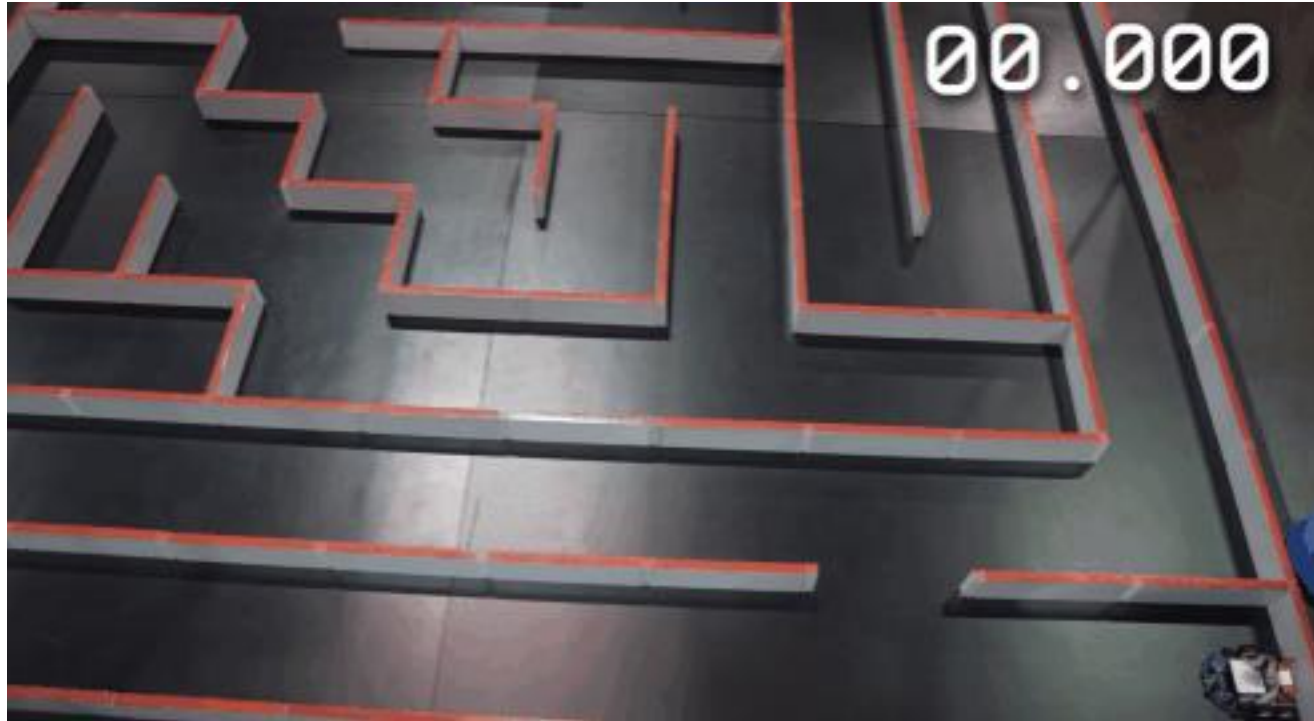
第七章 回溯法

(部 分 内 容 参 考 计 算 机 学 院
《 算 法 设 计 与 分 析 》)

吉林大学人工智能学院

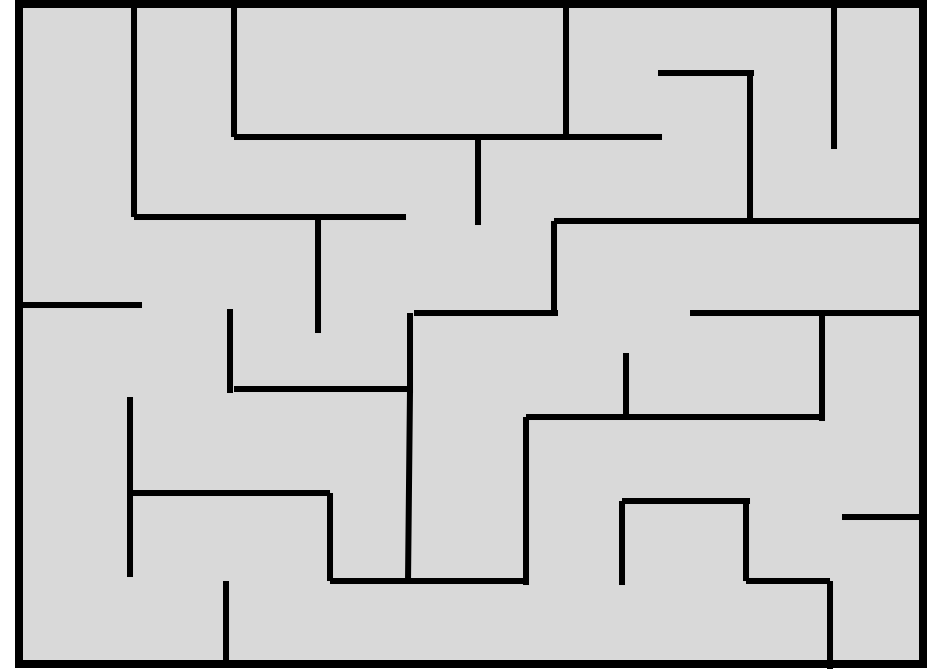
王元宏

迷宫游戏



迷宫游戏

入口

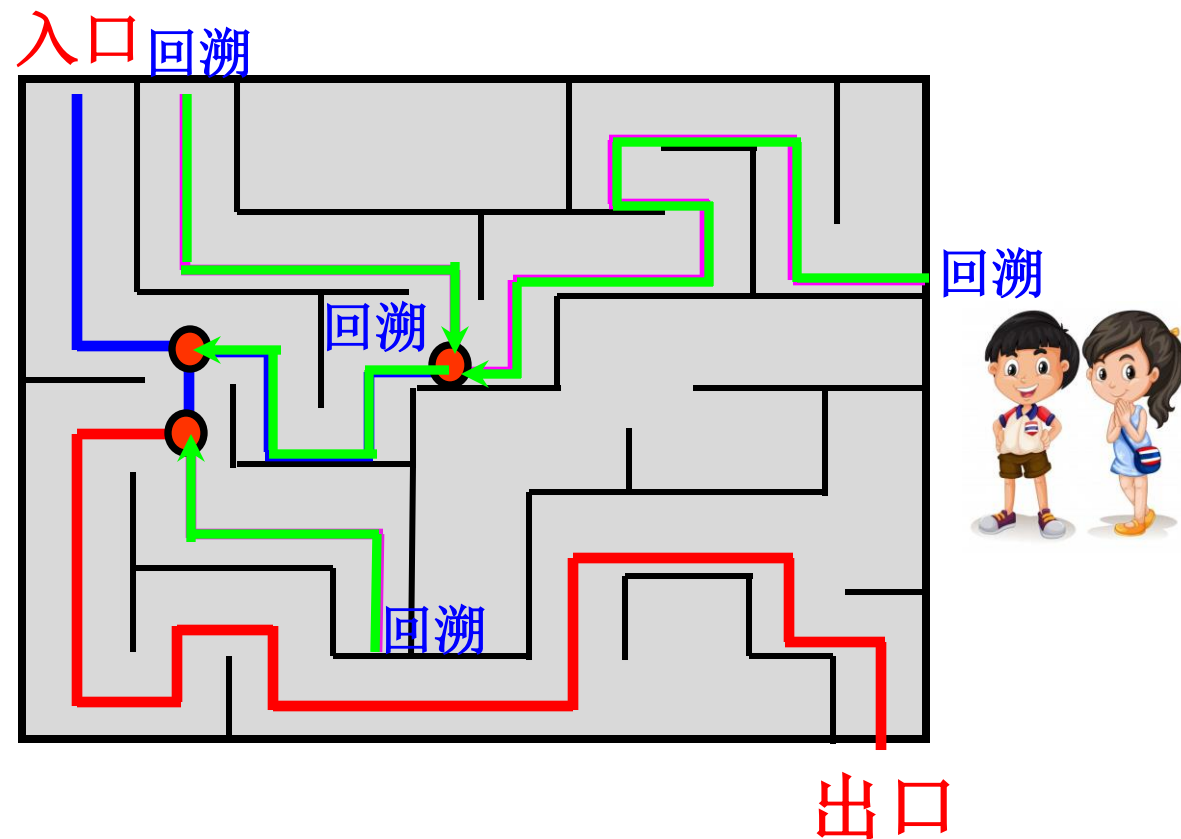


出口



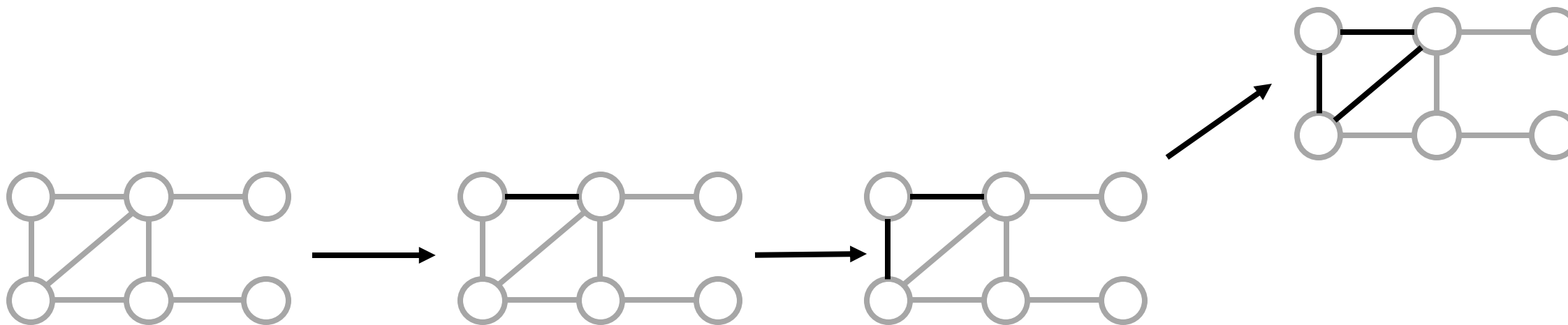
- 抽象成图中的搜索问题
 - 深度优先搜索 (DFS)
 - 广度优先搜索 (BFS)
 -

深度优先搜索

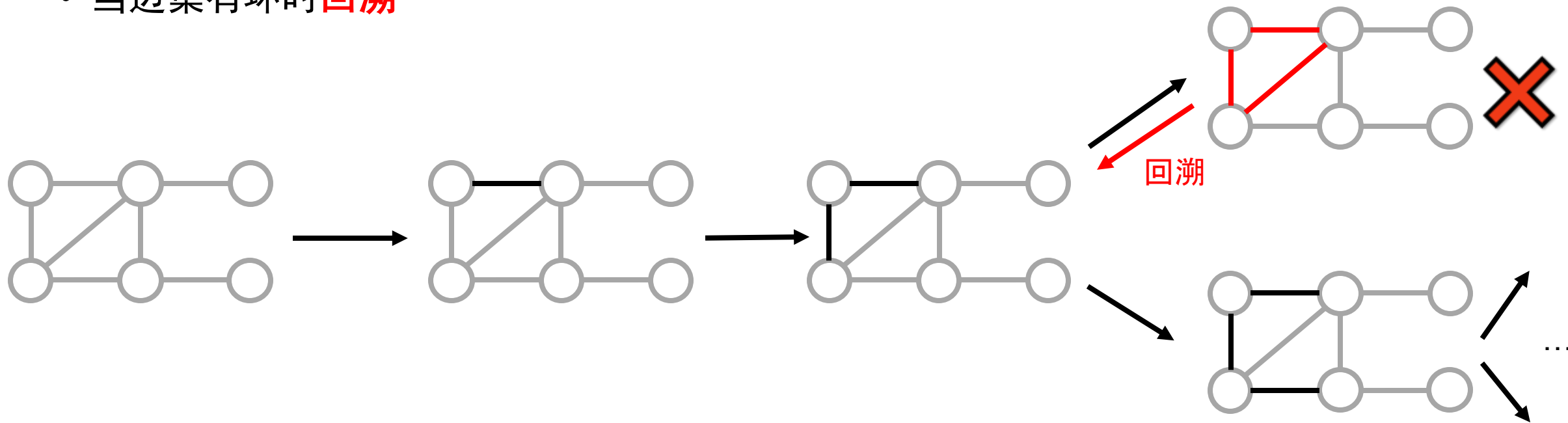


回溯法本质上是一种深度优先搜索算法

- 最小生成树问题
 - 算法每步添加一条边



- 最小生成树问题
 - 算法每步添加一条边
 - 当边集有环时**回溯**

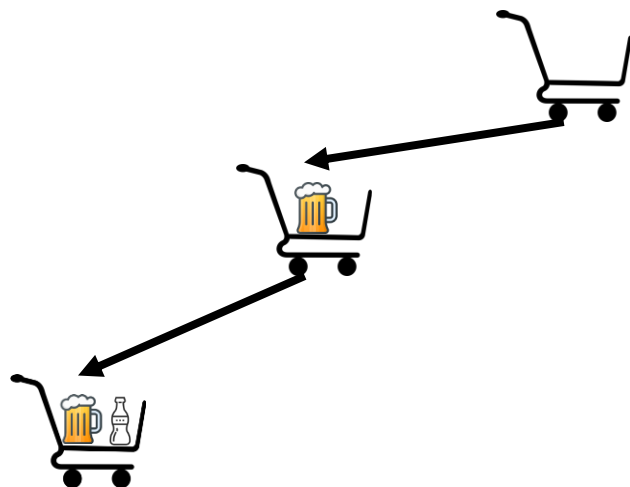


- 0/1背包问题
 - 一个体积大小为13的背包

商品	价格	体积
 啤酒	24	10
 汽水	2	3
 饼干	9	4
 面包	10	5
 牛奶	9	4

- 0/1背包问题

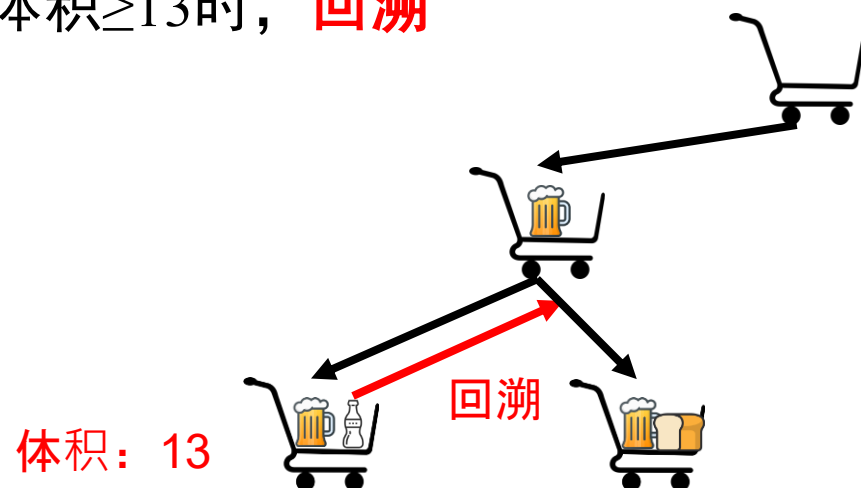
- 一个体积大小为13的背包
- 算法每步选择一个商品



商品	价格	体积
啤酒	24	10
汽水	2	3
饼干	9	4
面包	10	5
牛奶	9	4

- 0/1背包问题

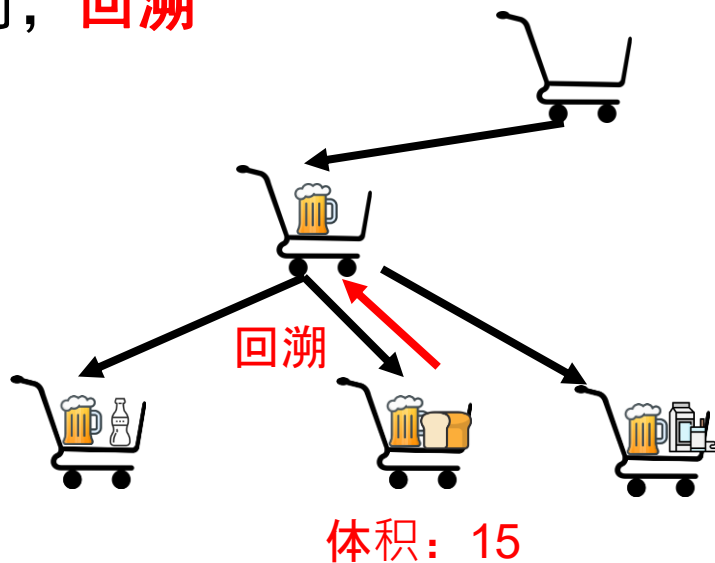
- 一个体积大小为13的背包
- 算法每步选择一个商品
- 总体积 ≥ 13 时，**回溯**



商品	价格	体积
啤酒	24	10
汽水	2	3
饼干	9	4
面包	10	5
牛奶	9	4

- 0/1背包问题

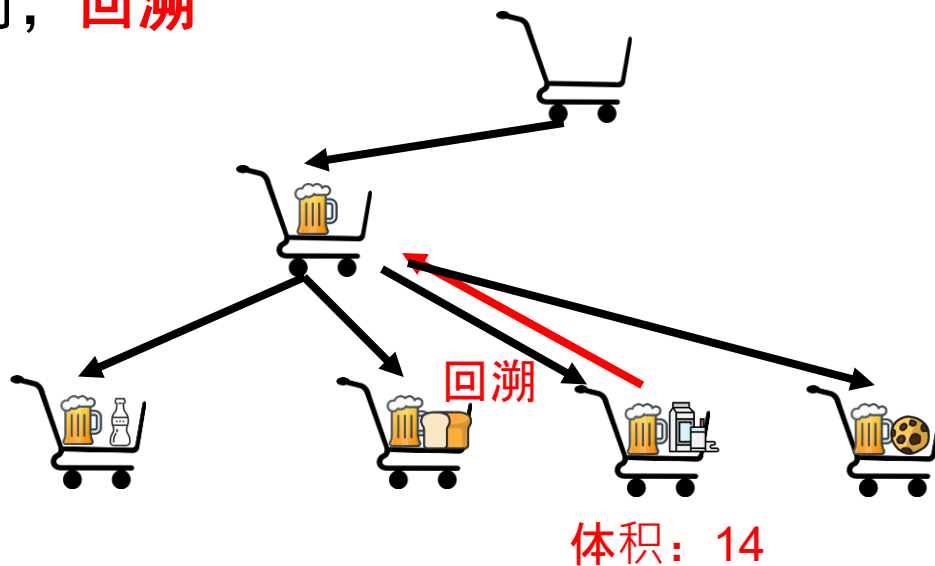
- 一个体积大小为13的背包
- 算法每步选择一个商品
- 总体积 ≥ 13 时，**回溯**



商品	价格	体积
啤酒	24	10
汽水	2	3
饼干	9	4
面包	10	5
牛奶	9	4

- 0/1背包问题

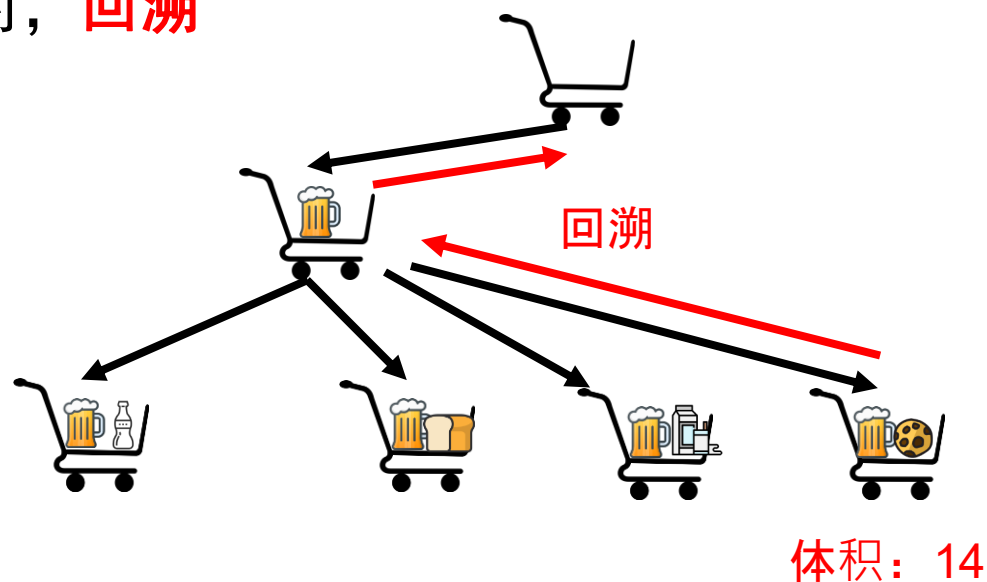
- 一个体积大小为13的背包
- 算法每步选择一个商品
- 总体积 ≥ 13 时，**回溯**



商品	价格	体积
啤酒	24	10
汽水	2	3
饼干	9	4
面包	10	5
牛奶	9	4

- 0/1背包问题

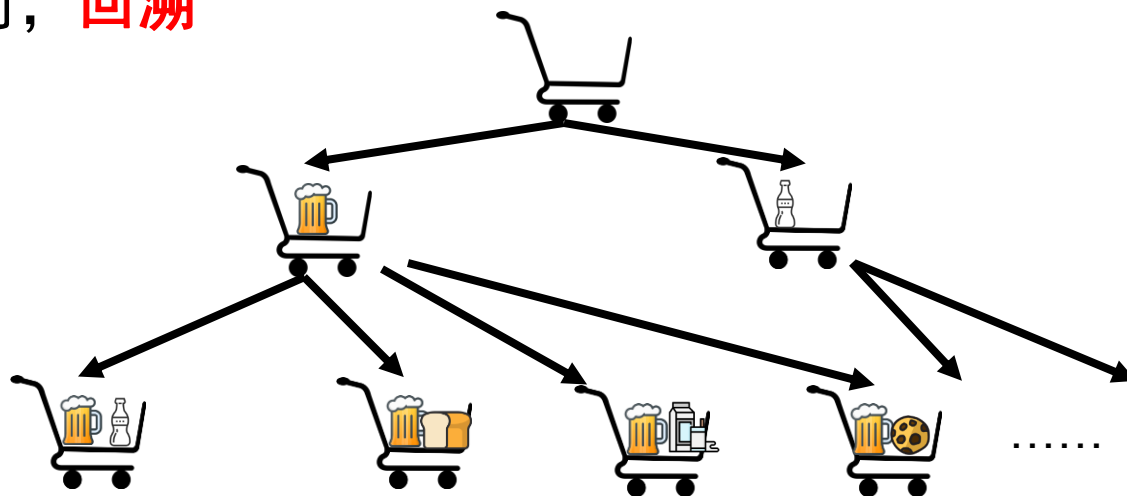
- 一个体积大小为13的背包
- 算法每步选择一个商品
- 总体积 ≥ 13 时，**回溯**



商品	价格	体积
啤酒	24	10
汽水	2	3
饼干	9	4
面包	10	5
牛奶	9	4

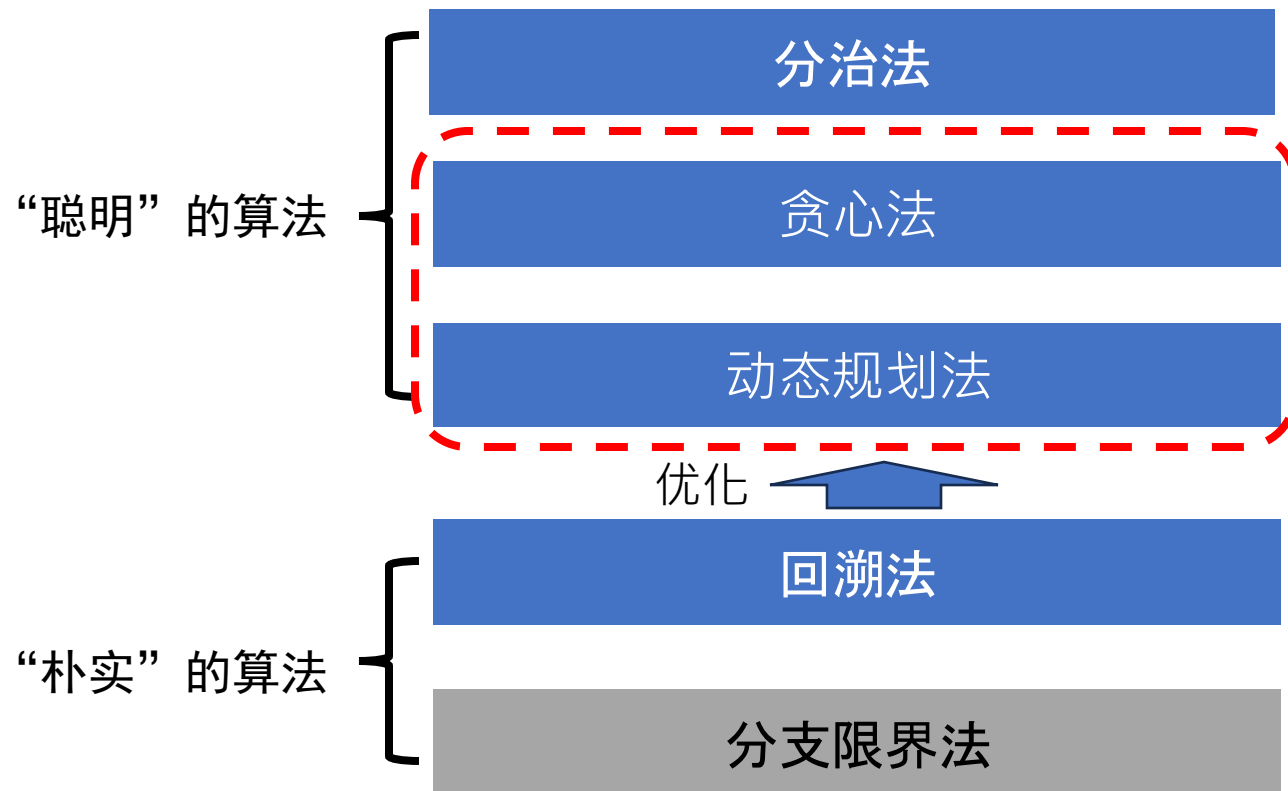
- 0/1背包问题

- 一个体积大小为13的背包
- 算法每步选择一个商品
- 总体积 ≥ 13 时，**回溯**



商品	价格	体积
啤酒	24	10
汽水	2	3
饼干	9	4
面包	10	5
牛奶	9	4

“朴实”的回溯法



George Pólya
(1887-1985)

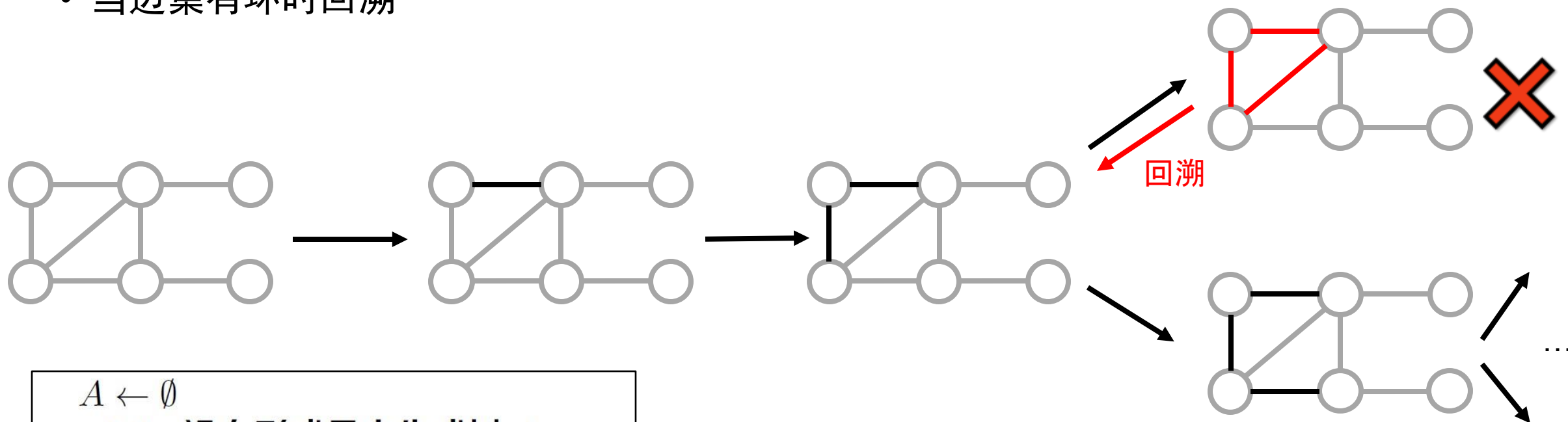
Pólya Enumeration Theorem
John von Neumann的导师

“这么精妙的算法是怎样设计出来的？我为什么没有想到这个算法呢？”



“四十岁后，不滞于物，草木竹石均可为剑。自此精修，渐进于无剑胜有剑之境。”

- 最小生成树问题
 - 算法每步添加一条边
 - 当边集有环时回溯



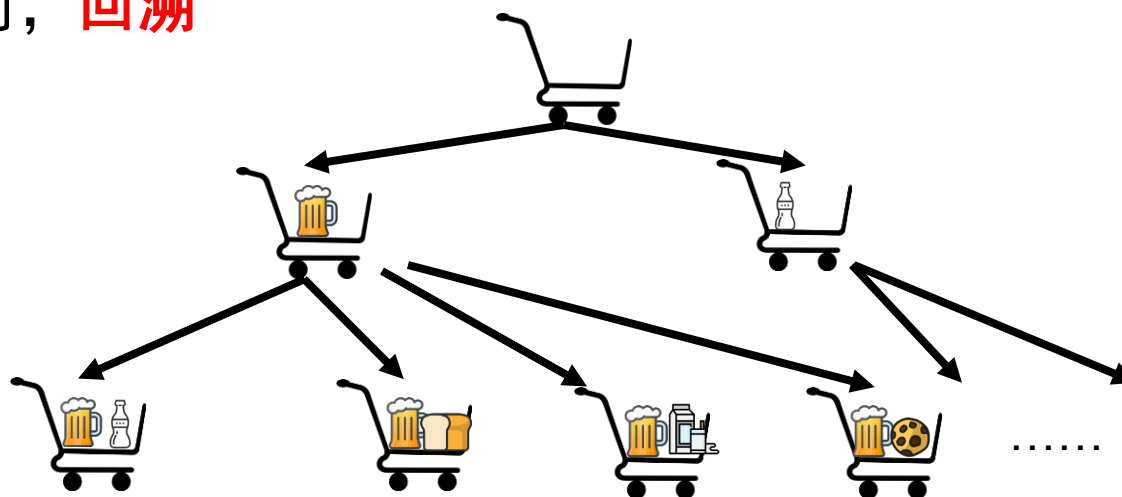
```
A ← ∅  
while 没有形成最小生成树 do  
    | 寻找A的安全边(u, v)  
    | A ← A ∪ (u, v)  
end  
return A
```

每次添加安全边！

- 0/1背包问题

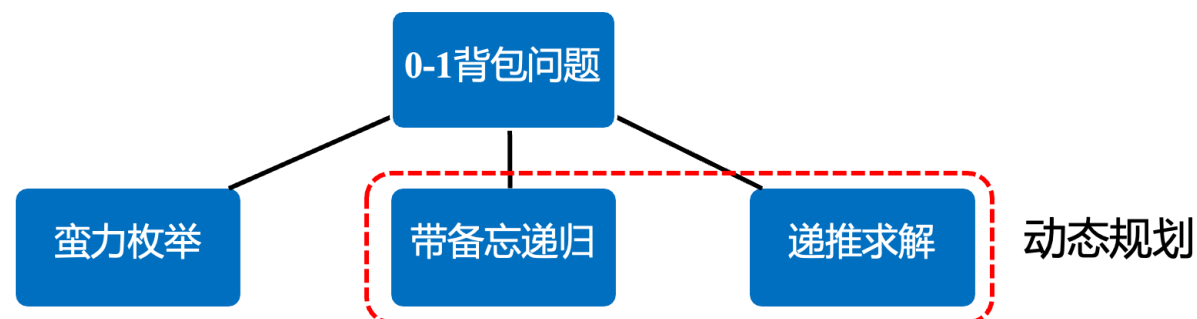
- 一个体积大小为13的背包
- 算法每步选择一个商品
- 总体积 ≥ 13 时，回溯

商品	价格	体积
啤酒	24	10
汽水	2	3
饼干	9	4
面包	10	5
牛奶	9	4



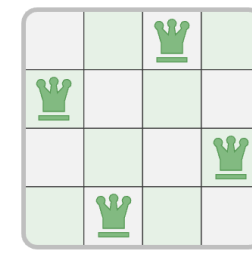
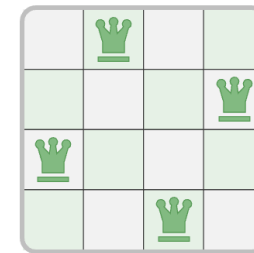
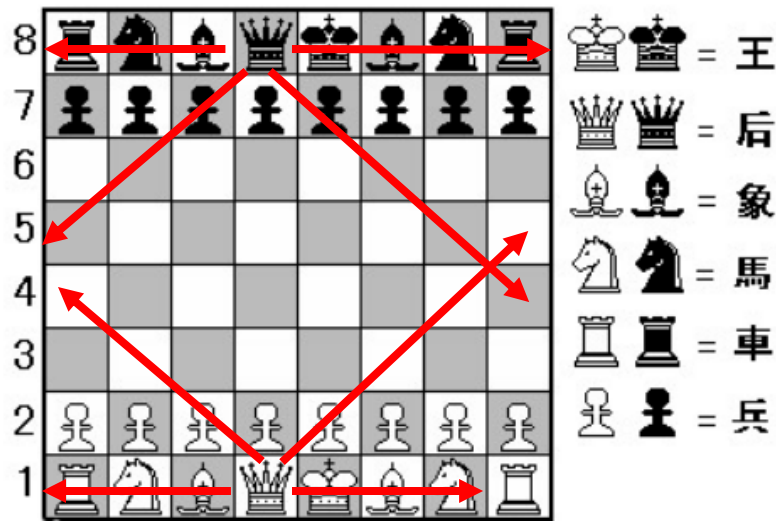
- 优化

- 记录子问题解
- 避免重复计算



4-皇后问题

- 国际象棋中，皇后是国际象棋棋局中实力最强的棋子。后可横直斜走，且格数不限
- 如何在棋盘中放置4个皇后，使她们不可相互攻击



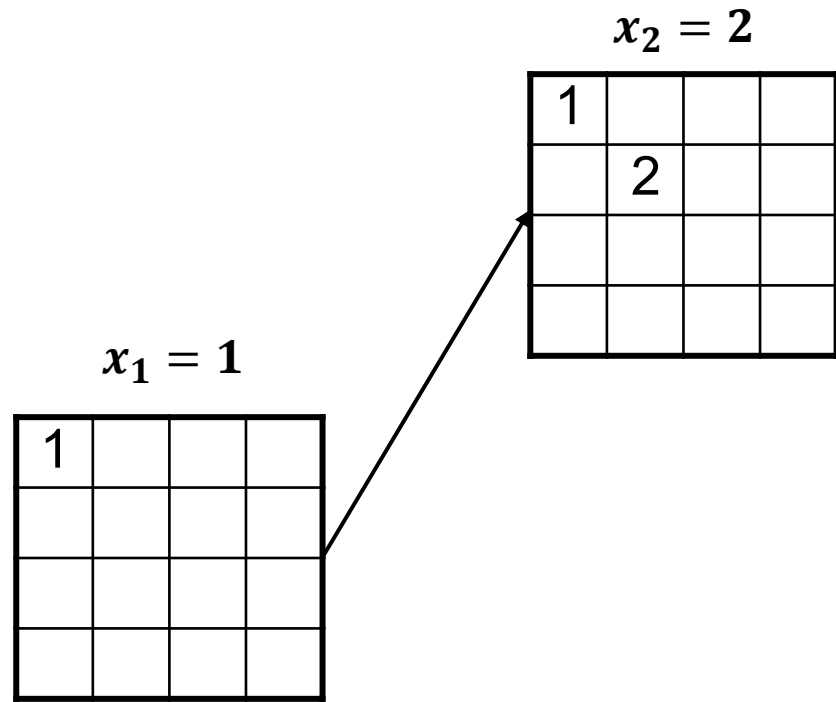
- 输入：棋盘大小 n
- 输出：4个皇后的位置 x_1, x_2, x_3, x_4 ， x_i 表示第 i 个皇后放在 (i, x_i) 处，使得

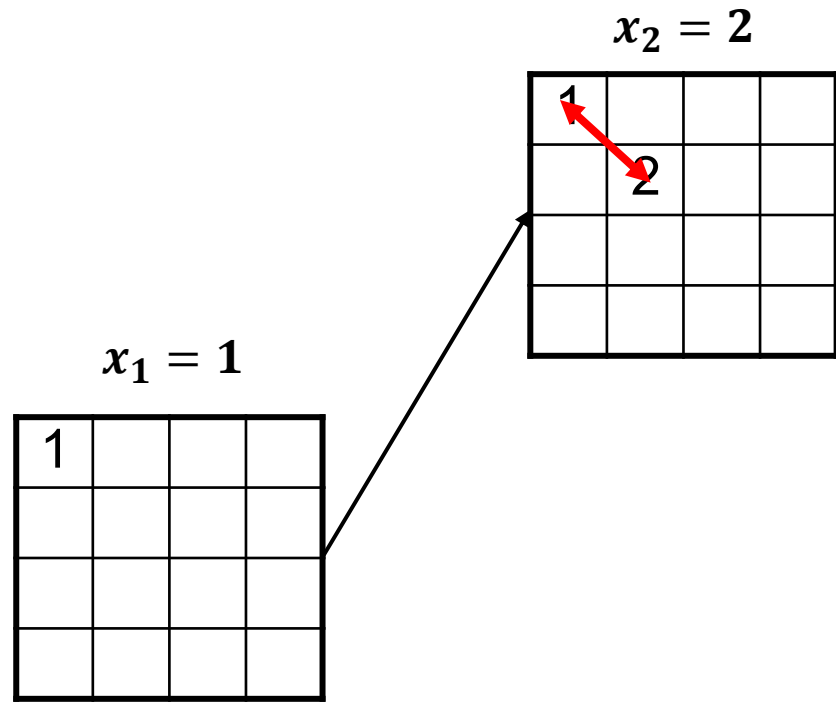
思考：为何能只用 x_i 表示皇后位置？

任意两个 x_i 都不相同, 且没有两个皇后可以在同一条斜角线上

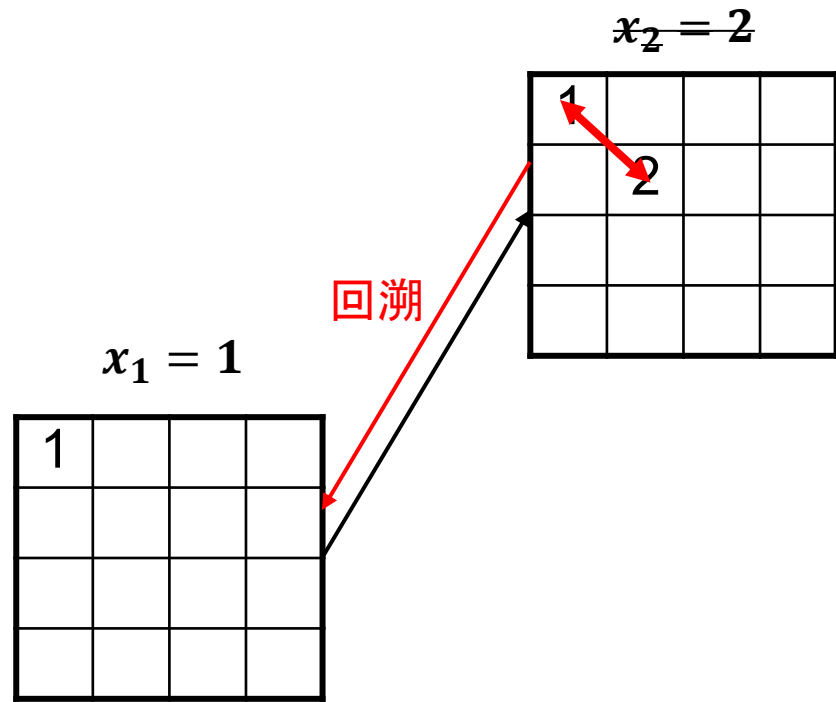
$$x_1 = 1$$

1			

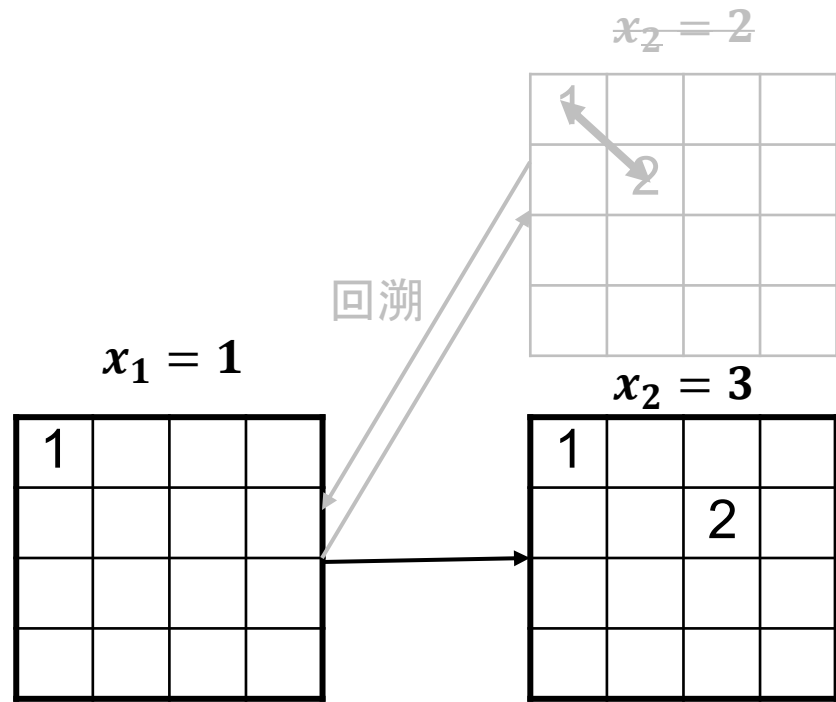




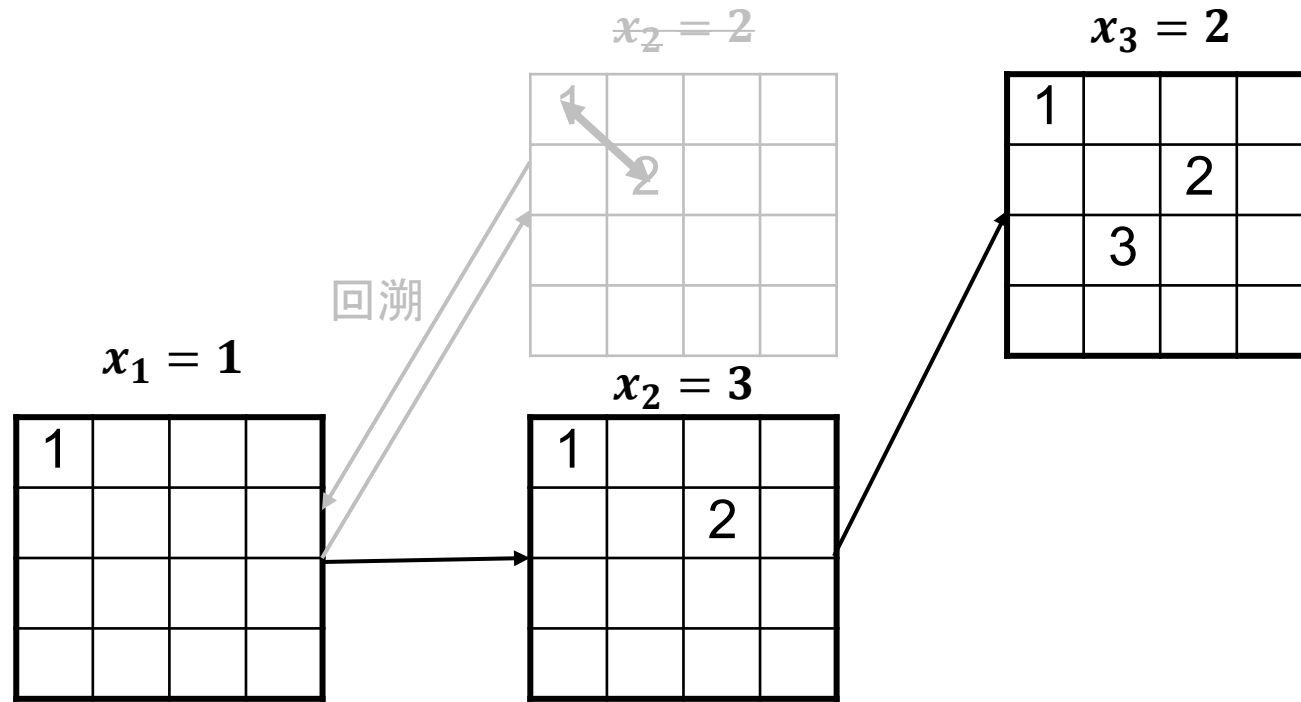
回溯法求解



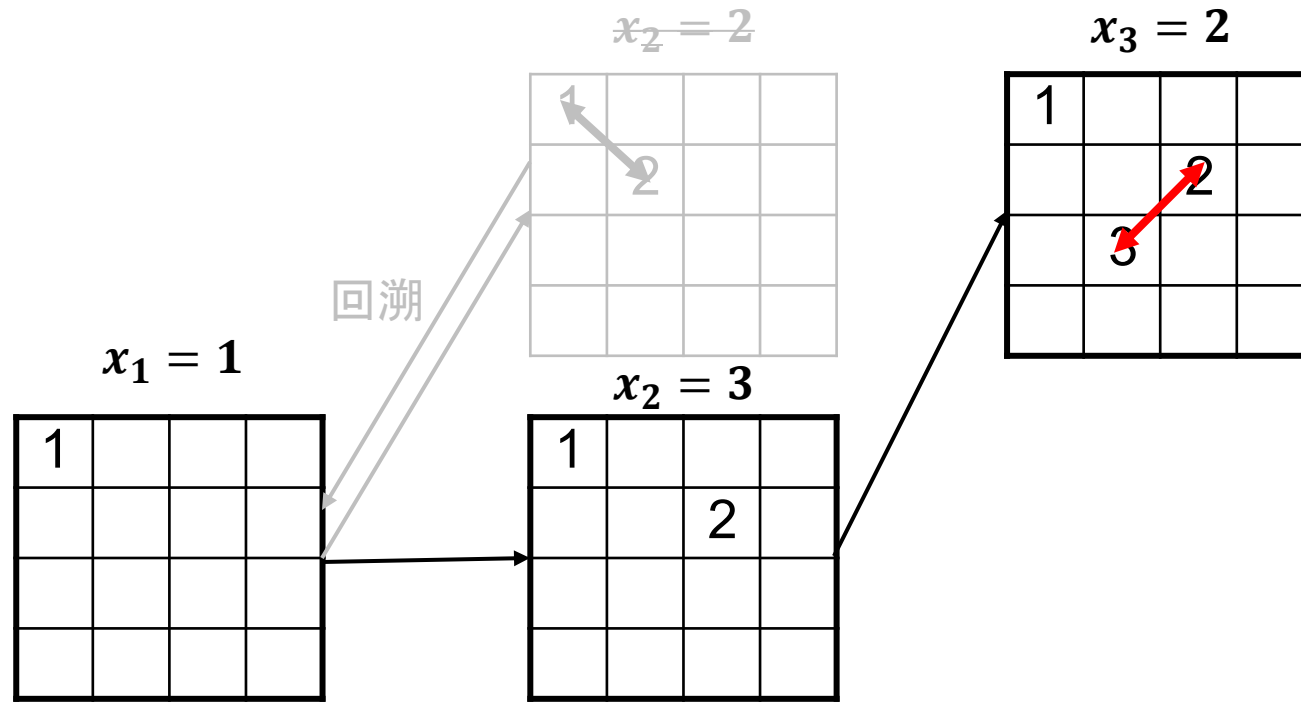
回溯法求解



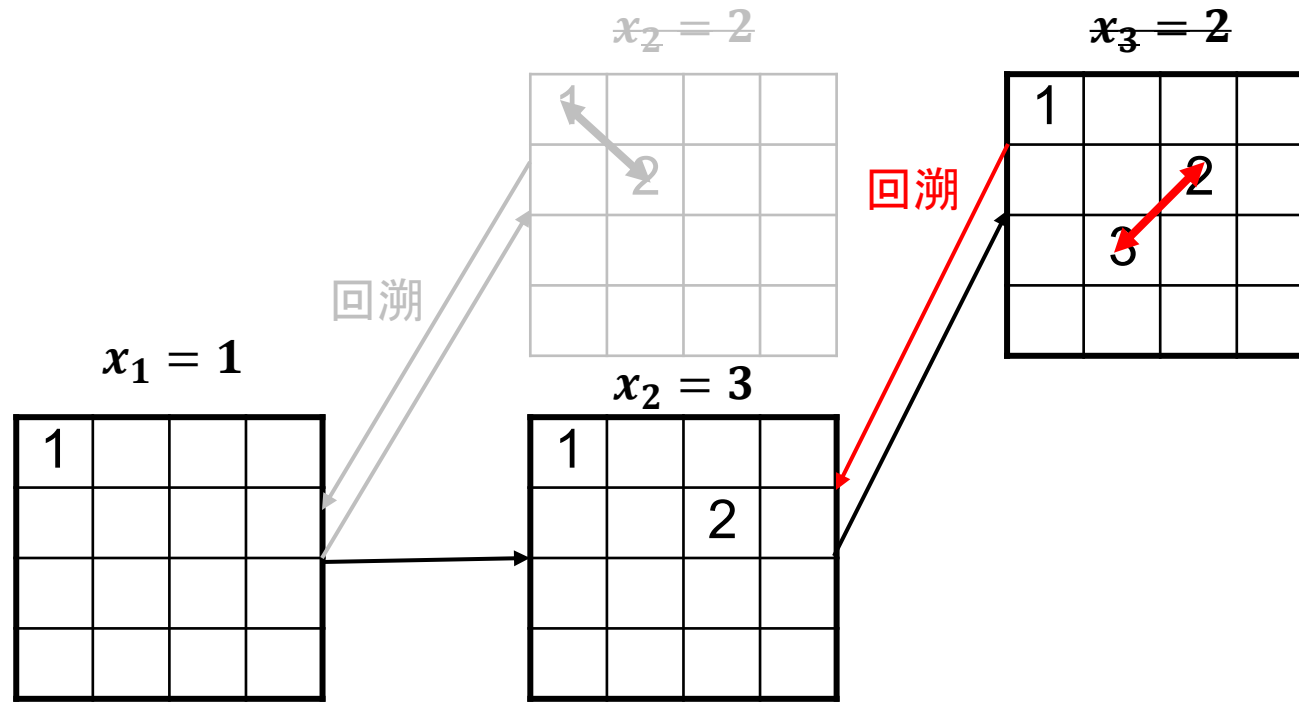
回溯法求解



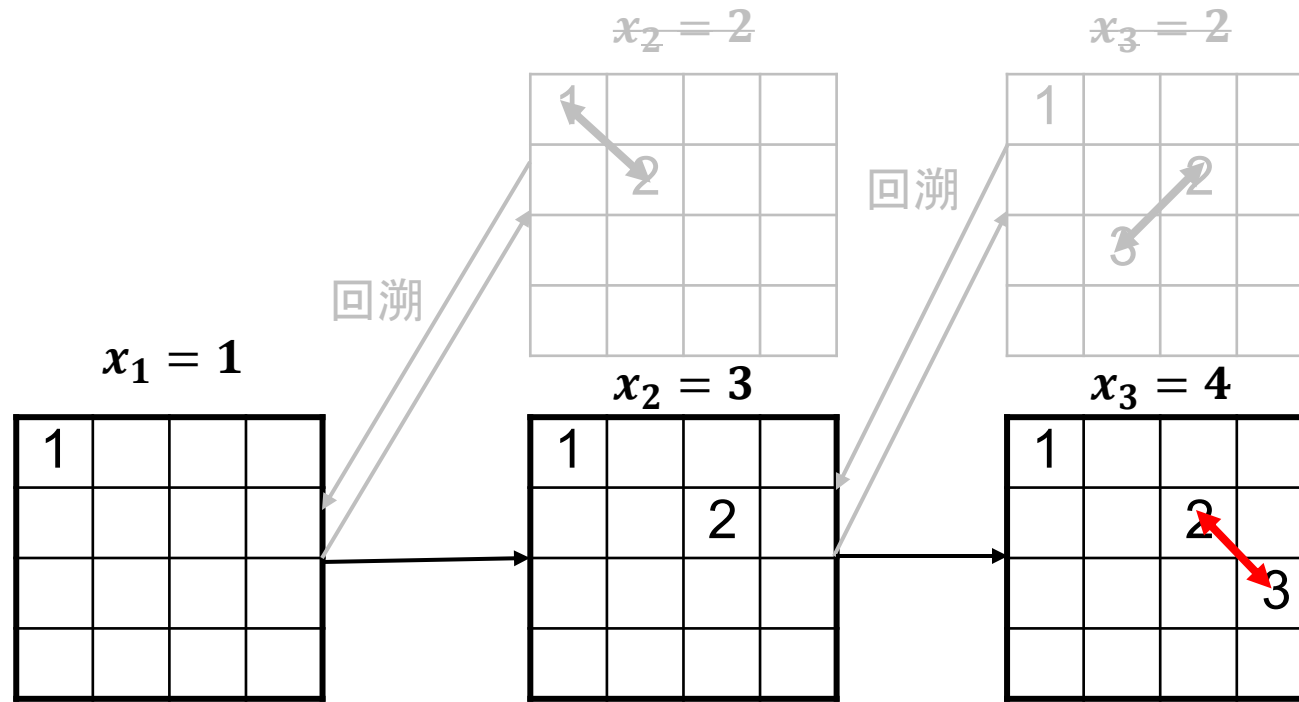
回溯法求解



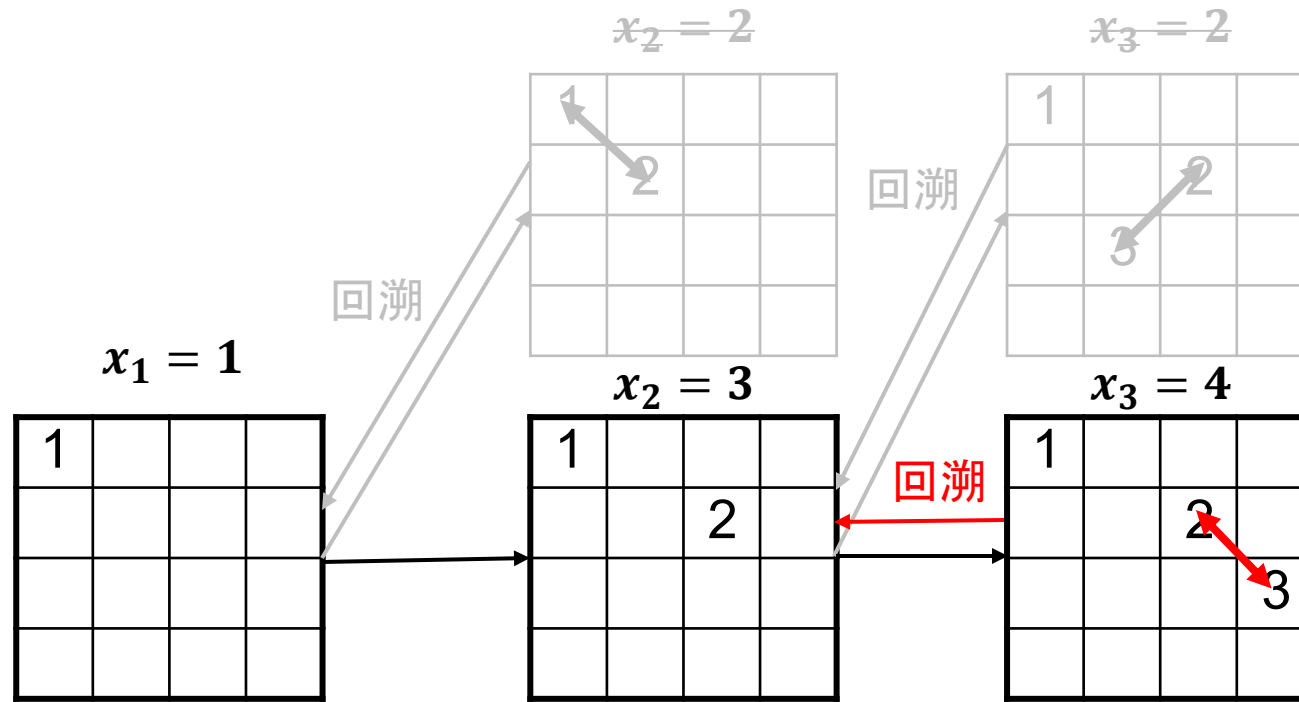
回溯法求解



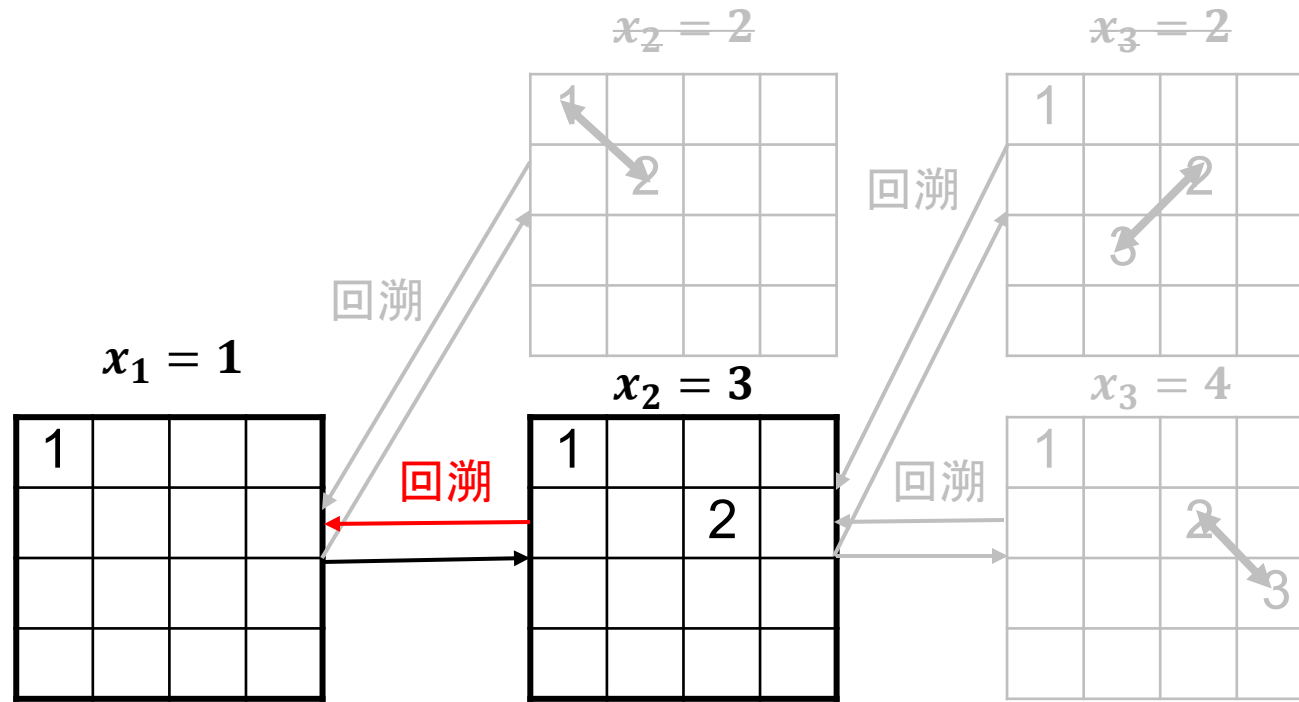
回溯法求解



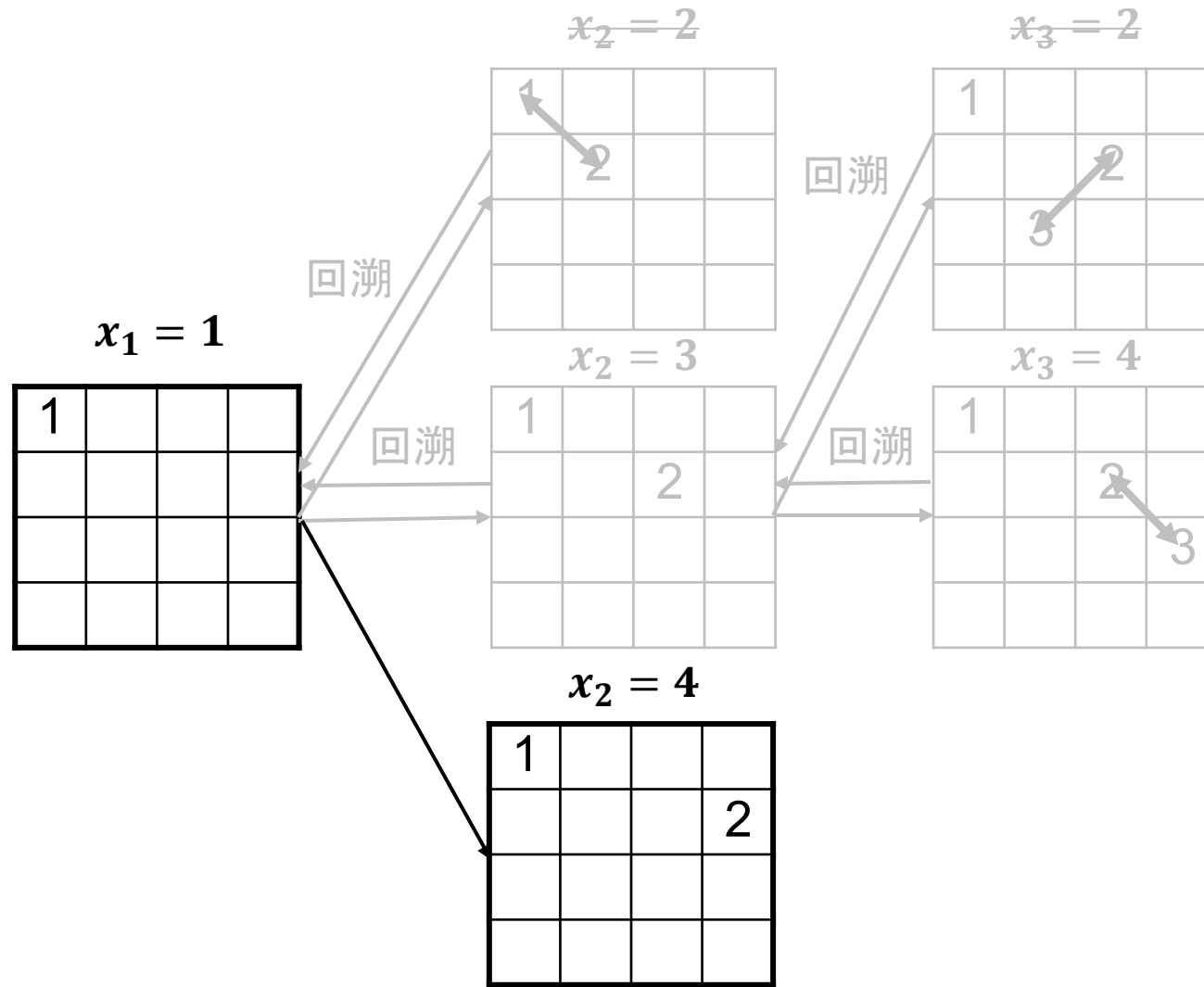
回溯法求解



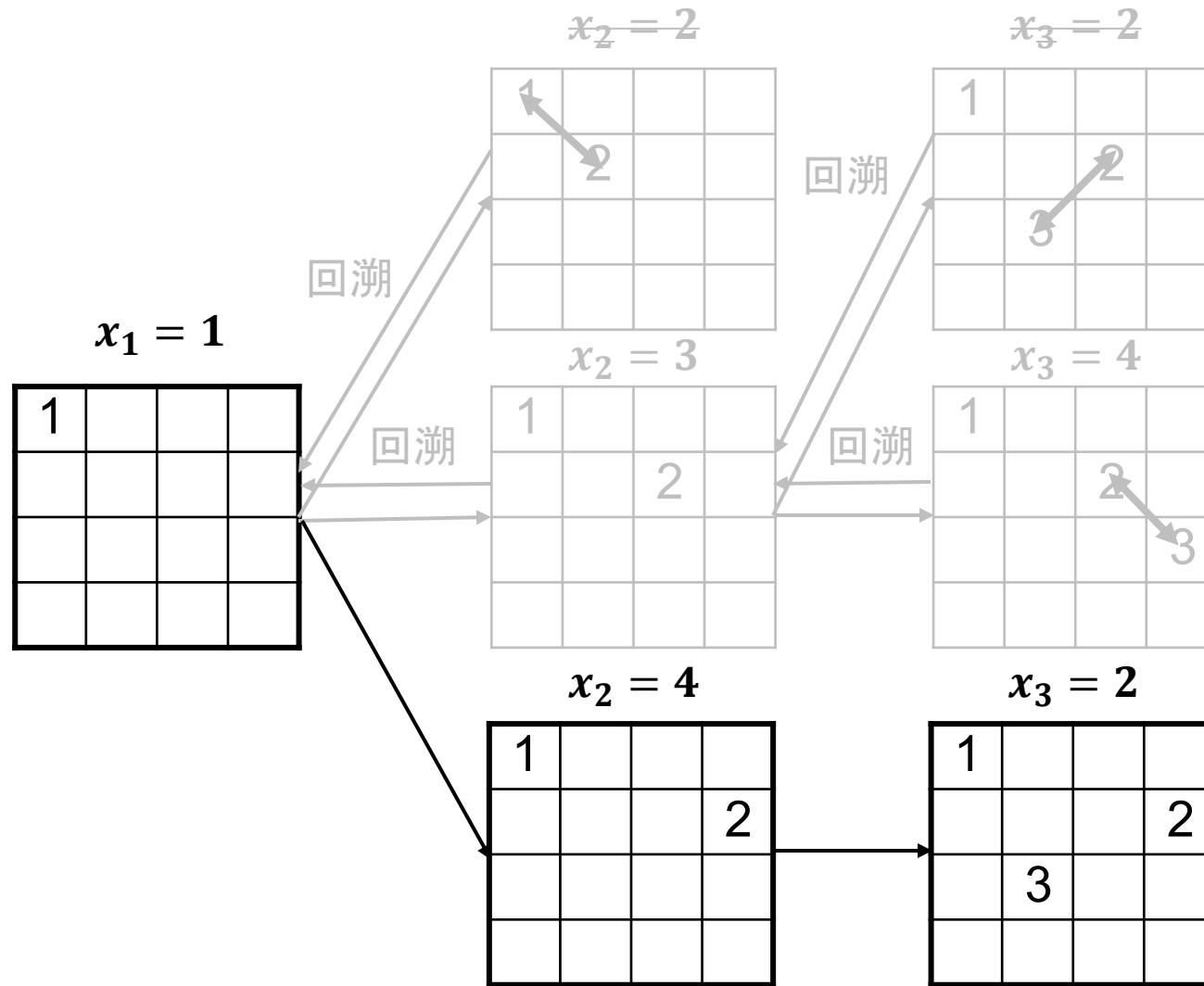
回溯法求解



回溯法求解



回溯法求解



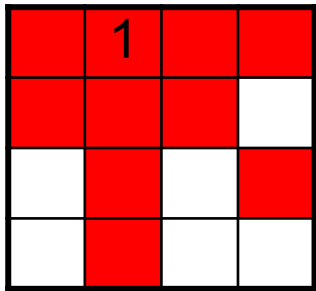


$$x_1 = 2$$

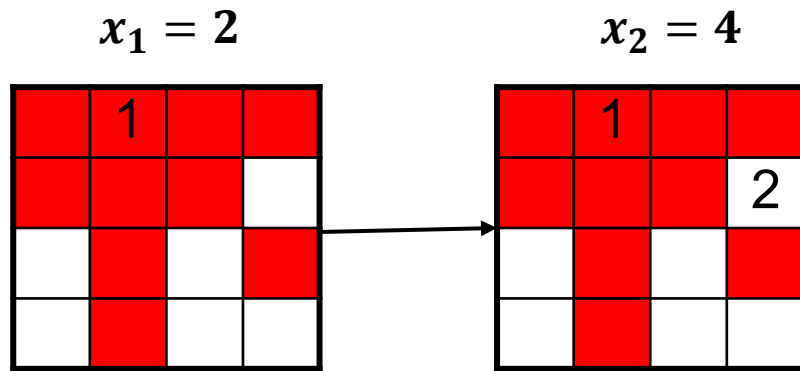
	1		

红色区域不能放

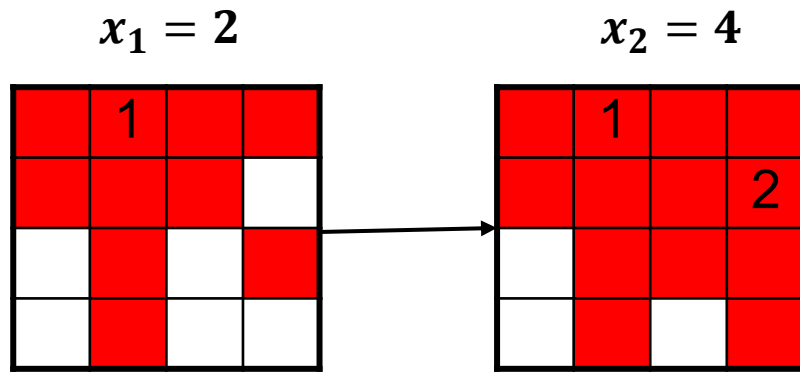
$$x_1 = 2$$



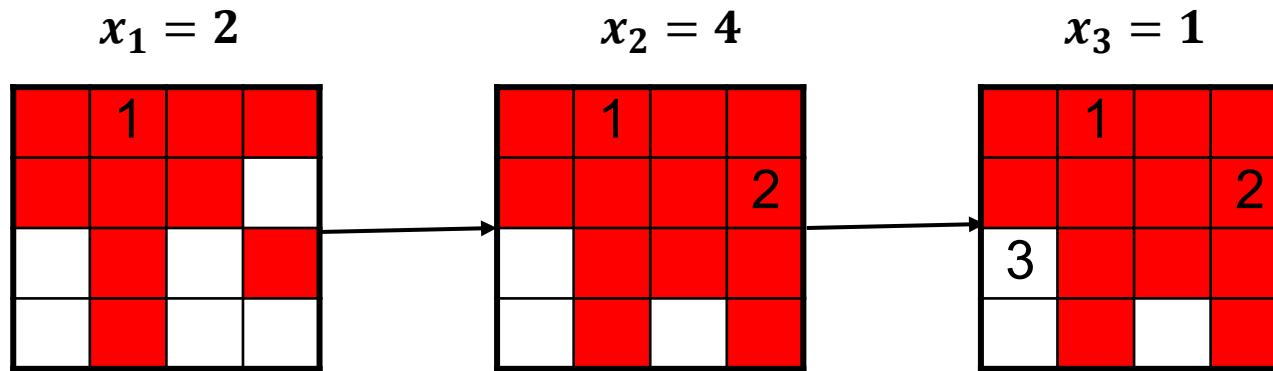
红色区域不能放



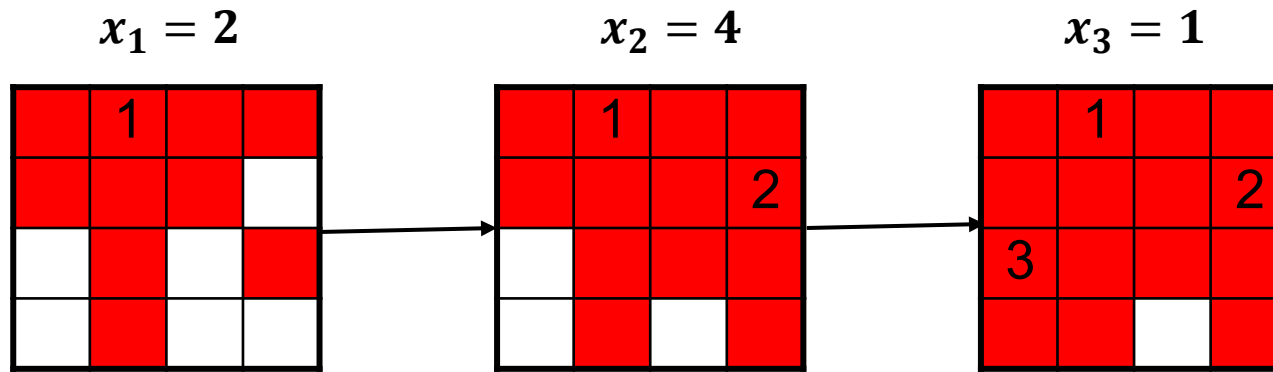
红色区域不能放



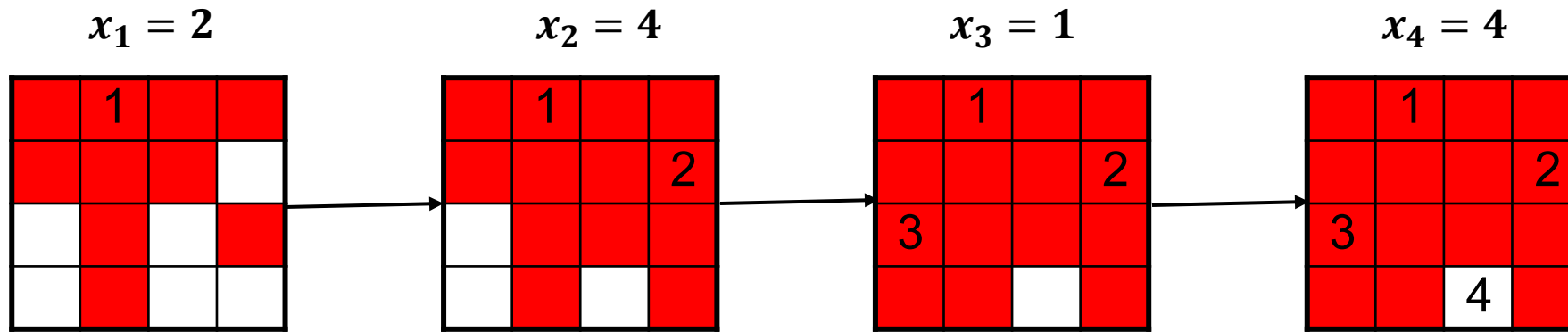
红色区域不能放



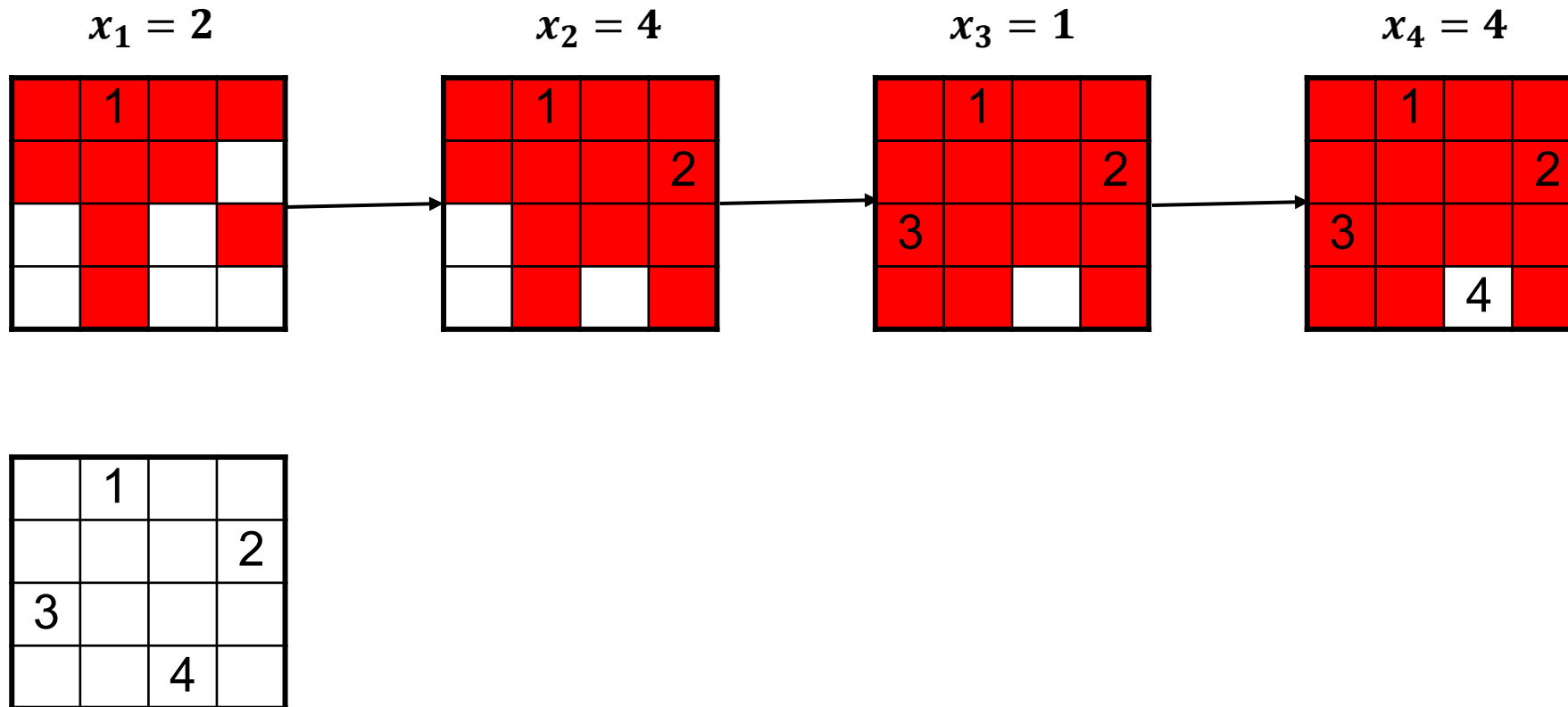
红色区域不能放



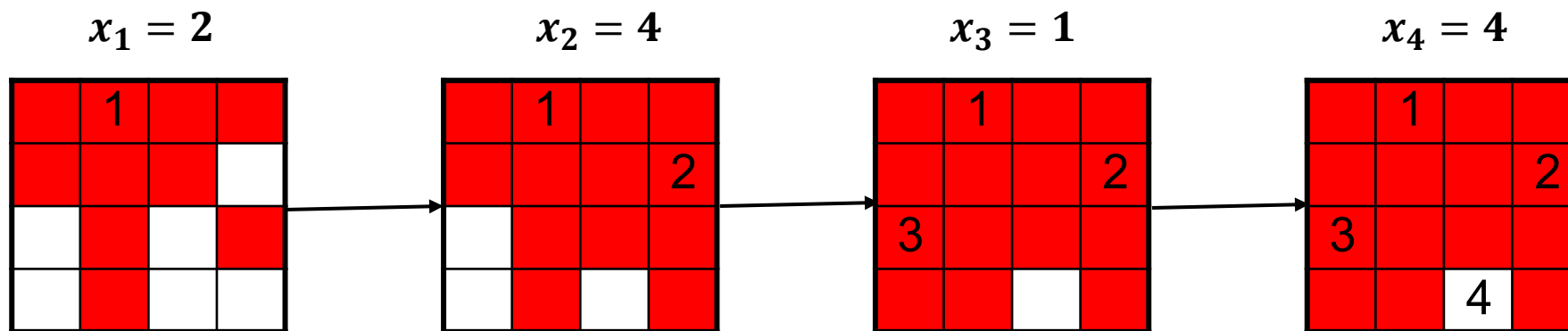
红色区域不能放



红色区域不能放



红色区域不能放



	1		
			2
3			
		4	

分治法:

- 子问题不独立!

贪心法:

- 局部最优 $\rightarrow$ 全局最优?

动态规划:

- 有后效性!
- 不存在重复子问题

别无他法，只能回溯!

- 7.1 一般方法
- 7.2 效率估计
- 7.3 n -皇后问题
- 7.4 子集和数问题
- 7.5 图着色问题
- 7.6 小结

- 7.1 一般方法
- 7.2 效率估计
- 7.3 n-皇后问题
- 7.4 子集和数问题
- 7.5 图着色问题
- 7.6 小结

- 适用于解决多阶段决策问题，也称为组合问题
 - 问题的解向量用 **元组** 来表示, 元素 x_i 通常取自于某个有穷集 $S_i, 1 \leq i \leq n$, n 表示问题规模
 - 固定长 n -元组 $(x_1, \dots, x_n)$
 - 可变长 k -元组 $(x_1, \dots, x_k), k < n$
 - 问题的目标：
 - 满足 **约束条件** 的一个解或多个解；通常用 **限界函数** 表示约束条件
 - 满足约束条件的最优解；此时需要定义 **目标函数**，使目标函数取极值的解是最优解
 - 问题满足 **多米诺** 性质

组合搜索

组合优化

- 若 $(x_1, x_2, \dots, x_{i+1})$ 满足约束，则 $(x_1, x_2, \dots, x_i)$ 也满足约束
- 逆反命题：若 $(x_1, x_2, \dots, x_i)$ 不满足约束，则 $(x_1, x_2, \dots, x_{i+1})$ 也不满足约束
- 回溯法的主要思想：
 - 每步决策时，检验正在构造的部分解向量 $(x_1, x_2, \dots, x_i)$ ，一旦发现不成立，则无需考虑后续 $x_{i+1}, \dots, x_n$ 的取值

思考：多米诺性质与最优子结构性质有什么异同？



问题的多米诺性质是回溯法提高算法效率的关键

- 输入：棋盘大小 n
- 输出：4个皇后的位置 x_1, x_2, x_3, x_4 ， x_i 表示第 i 个皇后放在 (i, x_i) 处，使得

任意两个 x_i 都不相同，且没有两个皇后可以在同一条斜角线上

- 解向量： $(x_1, x_2, x_3, x_4), x_i \in \{1, 2, 3, 4\}$
- 限界函数： $x_1 \neq x_2 \neq x_3 \neq x_4$ ，任意两个 x_i 不能同在同一条斜角线上
- 满足多米诺性质：若 $(x_1, x_2, \dots, x_{i+1})$ 满足约束，即皇后 $1, 2, \dots, i+1$ 相互不攻击，则皇后 $1, 2, \dots, i$ 相互也不攻击，即 $(x_1, x_2, \dots, x_i)$ 满足约束

- 显式约束：每个 x_i 的取值集合 S_i ，可能与问题实例有关，也可能无关
 - $x_i \geq 0$, $S_i = \{\text{所有非负实数}\}$
 - $x_i = 0 \text{ 或 } 1$, $S_i = \{0, 1\}$
- 隐式约束：描述了 x_i 彼此相关的情况，与问题实例有关

对应限界函数

- 输入：棋盘大小 n
- 输出：4个皇后的位置 x_1, x_2, x_3, x_4 ， x_i 表示第 i 个皇后放在 (i, x_i) 处，使得

任意两个 x_i 都不相同, 且没有两个皇后可以在同一条斜角线上

- 显式约束: $S_i = \{1, 2, 3, 4\}$, $1 \leq i \leq 4$
- 隐式约束: $x_1 \neq x_2 \neq x_3 \neq x_4$ ，任意两个 x_i 不能同在同一条斜角线上

- 显式约束：每个 x_i 的取值集合 S_i ，可能与问题实例有关，也可能无关
 - $x_i \geq 0$ ， $S_i = \{\text{所有非负实数}\}$
 - $x_i = 0$ 或 1 ， $S_i = \{0, 1\}$
 - 4-皇后问题： $S_i = \{1, 2, 3, 4\}$
- 隐式约束：描述了 x_i 彼此相关的情况，与问题实例有关
- 解空间：满足显式约束条件的所有元组（容易枚举出）
- 可行解：解空间中满足隐式约束条件的元组

4-皇后问题

- 输入：棋盘大小 n
- 输出：4个皇后的位置 x_1, x_2, x_3, x_4 ， x_i 表示第 i 个皇后放在 (i, x_i) 处，使得

任意两个 x_i 都不相同, 且没有两个皇后可以在同一条斜角线上

- 显式约束: $S_i = \{1, 2, 3, 4\}$, $1 \leq i \leq 4$
- 隐式约束: $x_1 \neq x_2 \neq x_3 \neq x_4$ ，任意两个 x_i 不能同在同一条斜角线上
- 解空间: $\{1, 2, 3, 4\}^4$ 大小为 4^4 或者 $\{\{1, 2, 3, 4\}, \{1, 2, 4, 3\}, \{1, 3, 2, 4\}, \dots\}$

大小为4!

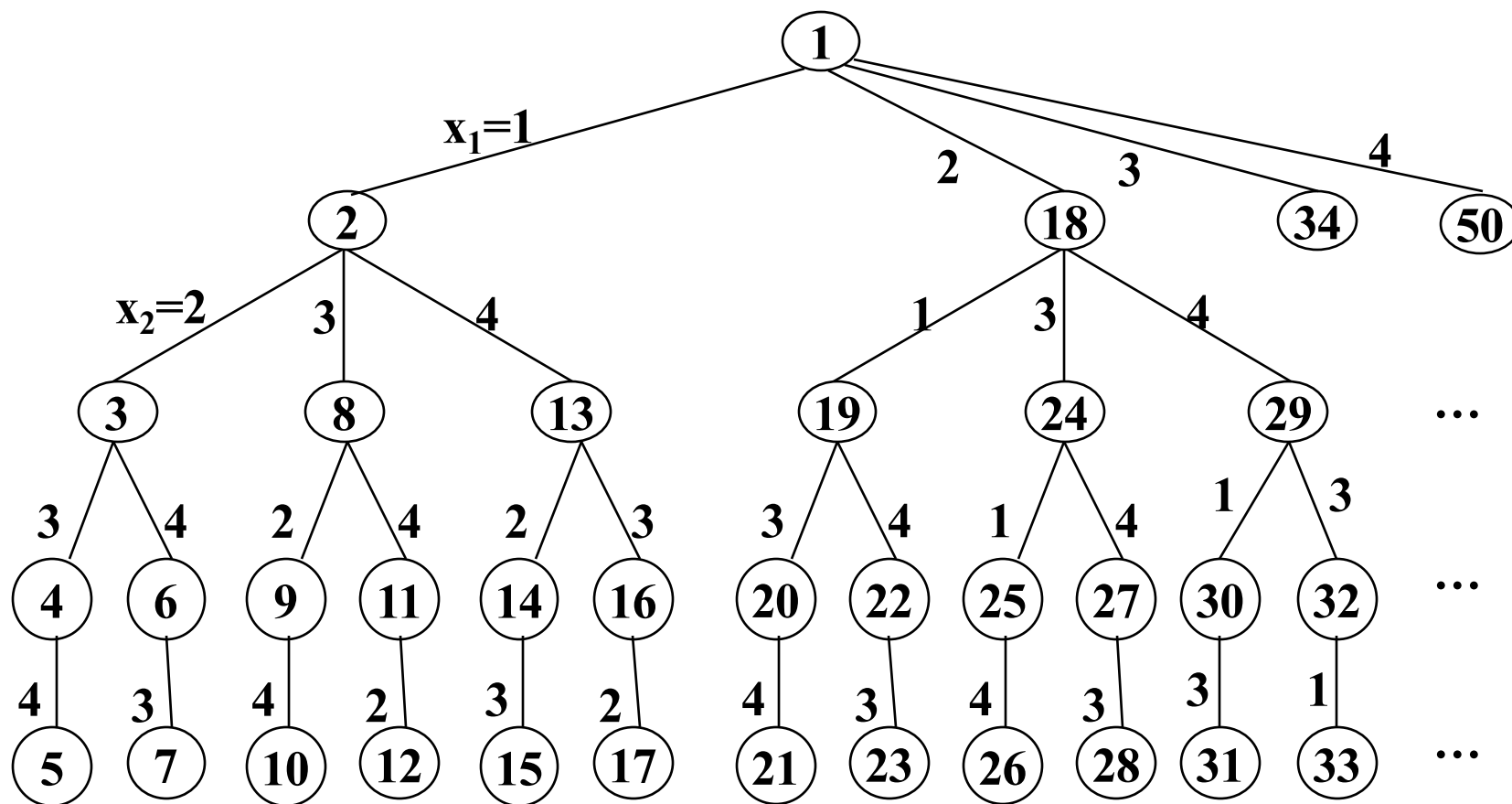
- 可行解：例如

	1		
			2
3			
		4	

- 显式约束：每个 x_i 的取值集合 S_i ，可能与问题实例有关，也可能无关
 - $x_i \geq 0$ ， $S_i = \{\text{所有非负实数}\}$
 - $x_i = 0$ 或 1 ， $S_i = \{0, 1\}$
 - 4-皇后问题： $S_i = \{1, 2, 3, 4\}$
- 隐式约束：描述了 x_i 彼此相关的情况，与问题实例有关
- 解空间：满足显式约束条件的所有元组（容易枚举出）
- 可行解：解空间中满足隐式约束条件的元组
- 状态空间树：解空间构成的树

4-皇后问题的状态空间树

- $n=4$ 时，叶结点个数= $4! = 24$ ，解空间是从根结点到叶结点的所有路径

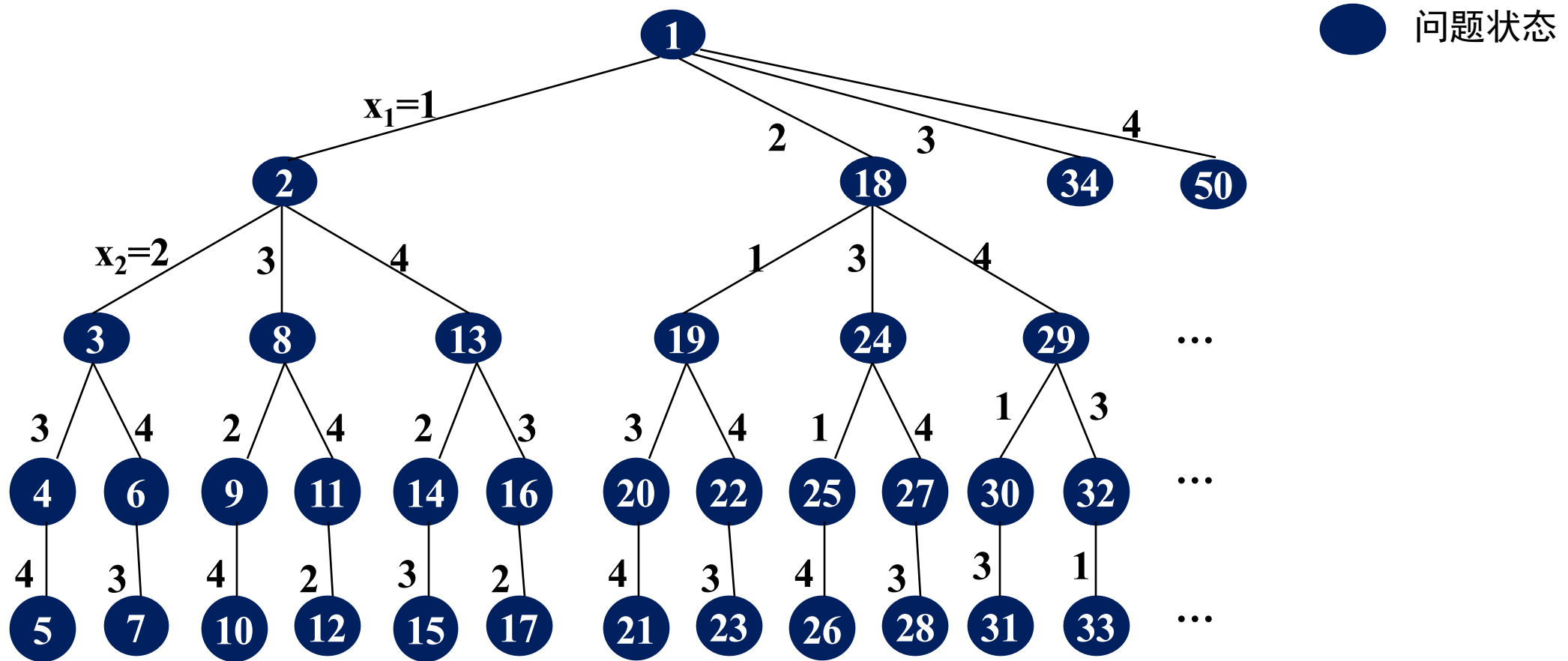


- 状态空间树：解空间构成的树
 - 问题状态：状态空间树中的所有结点
 - 解状态X：从根结点到X确定了解空间中一个元组
 - 答案状态X：从根结点到X确定了问题一个可行解

回溯法解决问题的过程就是在状态空间树上搜索答案状态结点的过程

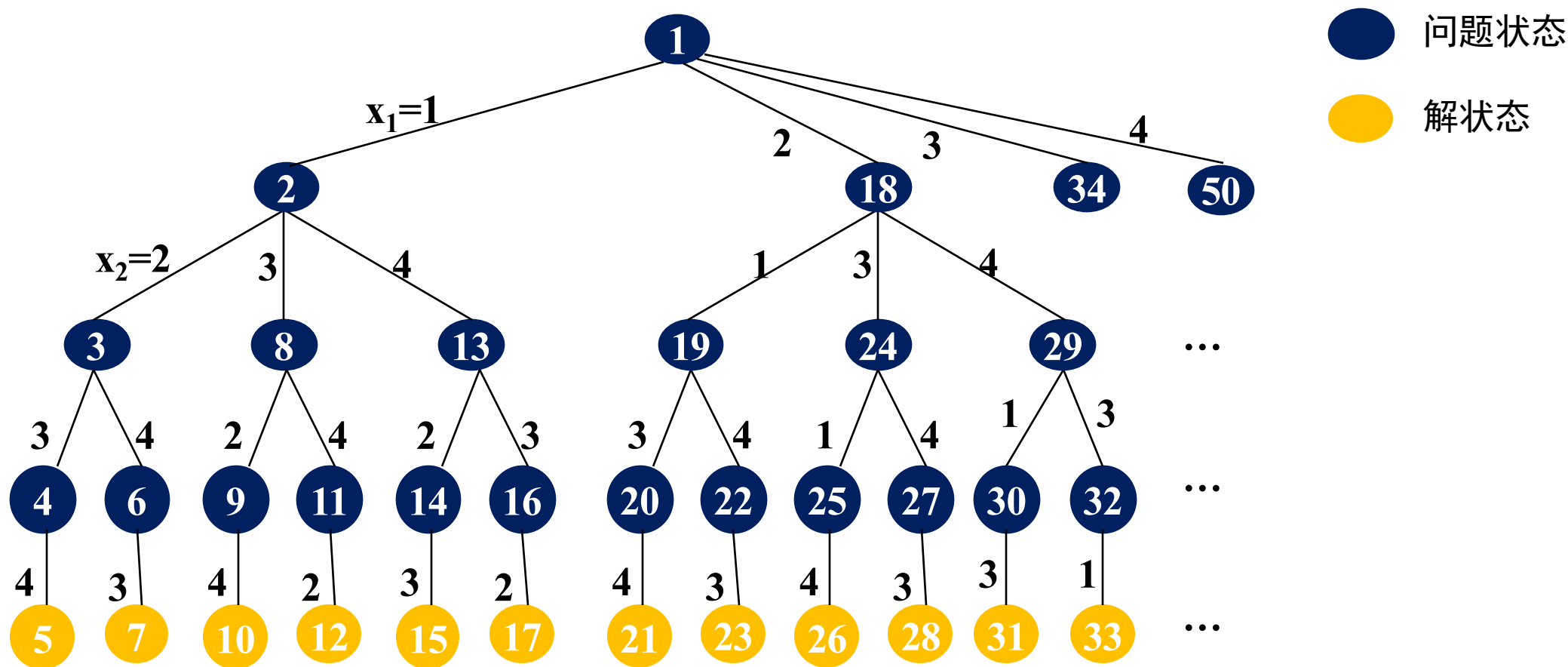
4-皇后问题的状态空间树

- $n=4$ 时，叶结点个数= $4! = 24$ ，解空间是从根结点到叶结点的所有路径



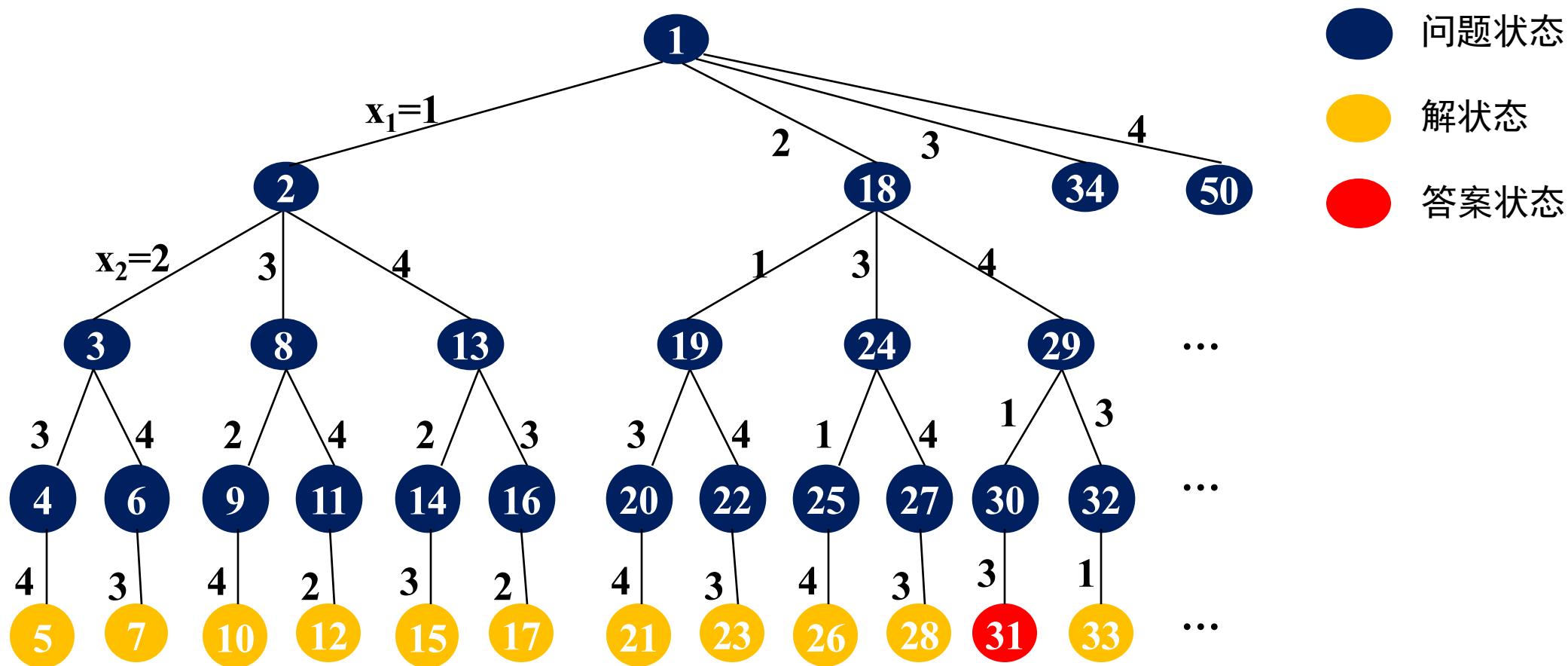
4-皇后问题的状态空间树

- $n=4$ 时，叶结点个数= $4! = 24$ ，解空间是从根结点到叶结点的所有路径



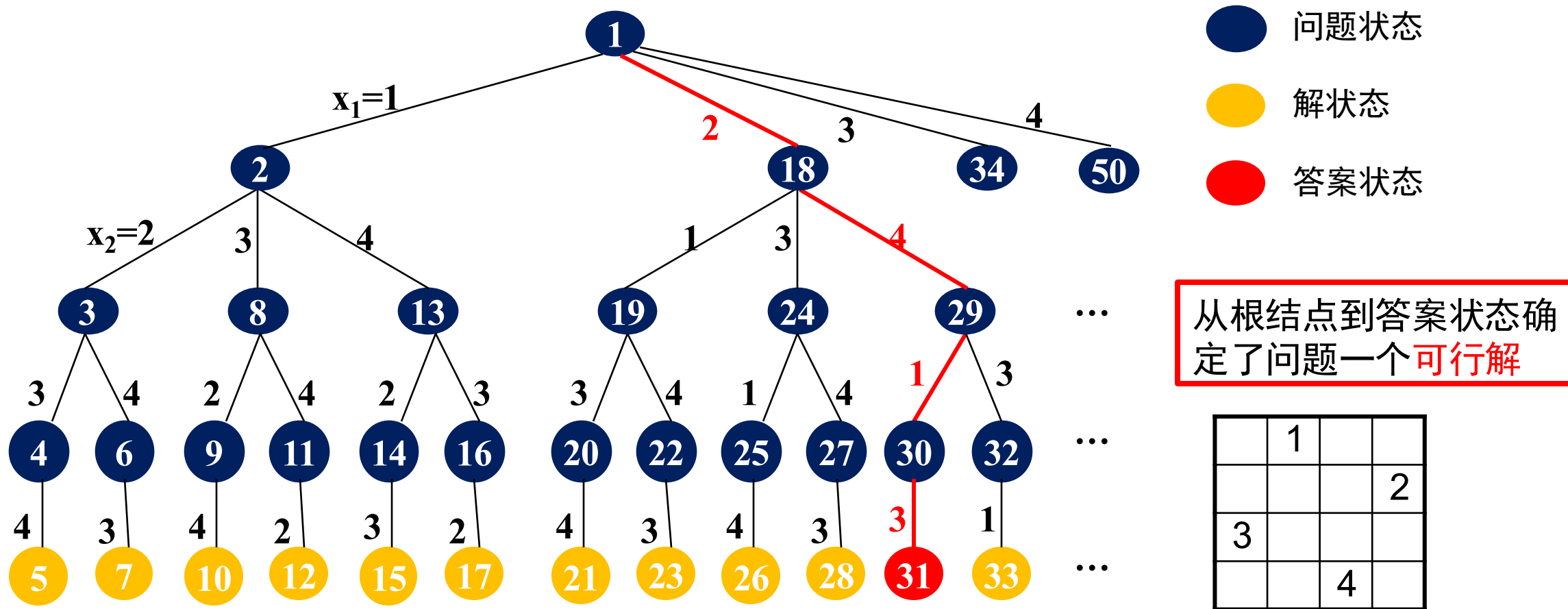
4-皇后问题的状态空间树

- $n=4$ 时，叶结点个数= $4! = 24$ ，解空间是从根结点到叶结点的所有路径



4-皇后问题的状态空间树

- $n=4$ 时，叶结点个数= $4! = 24$ ，解空间是从根结点到叶结点的所有路径



回溯法解决问题的过程就是在状态空间树上搜索答案状态结点的过程

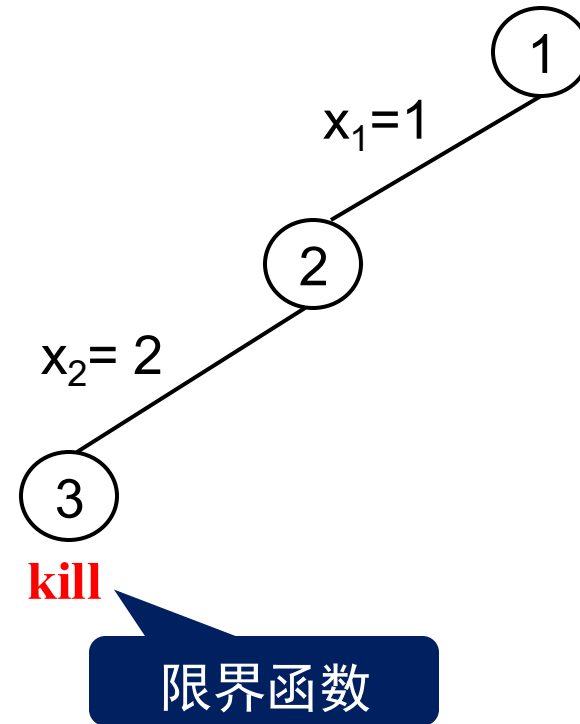
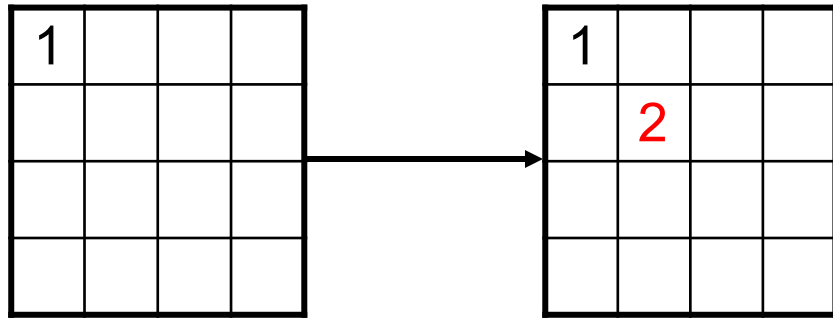
- 搜索状态空间树等同于构造状态空间树
- **活结点**：自己已经生成而其孩子结点还没有全部生成的结点
- **E-结点** (Extended-结点)：当前正在生成其孩子结点的活结点
- **死结点**：不再进一步扩展或者其孩子结点已全部生成的结点

- 第一种构造方法
 - 当前的E-结点R一旦生成一个新的孩子结点C, 这个C结点就变成一个新的E-结点, 当检测完了子树C后, R结点就再次成为E-结点, 生成下一个孩子结点
 - 该方法也称为**深度优先**生成法, 对应**回溯**法
- 第二种构造方法:
 - 一个E-结点一直保持到变成死结点为止, 对应**分枝限界**法
 - 当活结点用队列保存时, 该方法也称为**宽度优先**生成法
 - 当活结点用栈保存时, 该方法也称为**D-检索**生成法

构造过程中使用**限界函数**避免无效生成

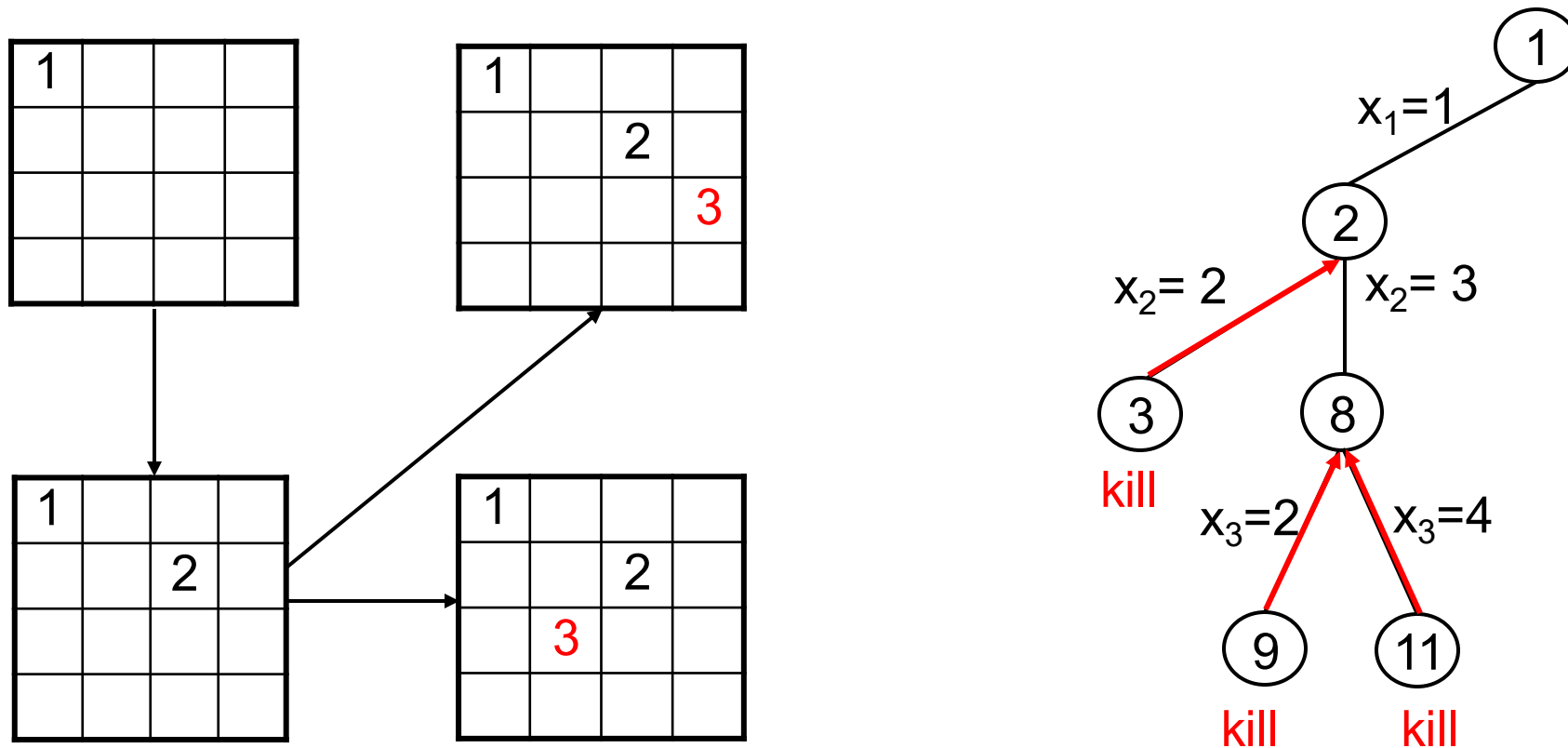
4-皇后问题的状态空间树构造

- 开始把根结点作为唯一的活结点, 根结点就成为E-结点而且路径为($()$); 接着生成孩子结点, 那么结点2被生成, 这条路径为(1), 即把皇后1放在第1列上
- 结点2变成E-结点, 它再生成结点3, 路径变为($1, 2$), 结点3被杀死, 此时回溯



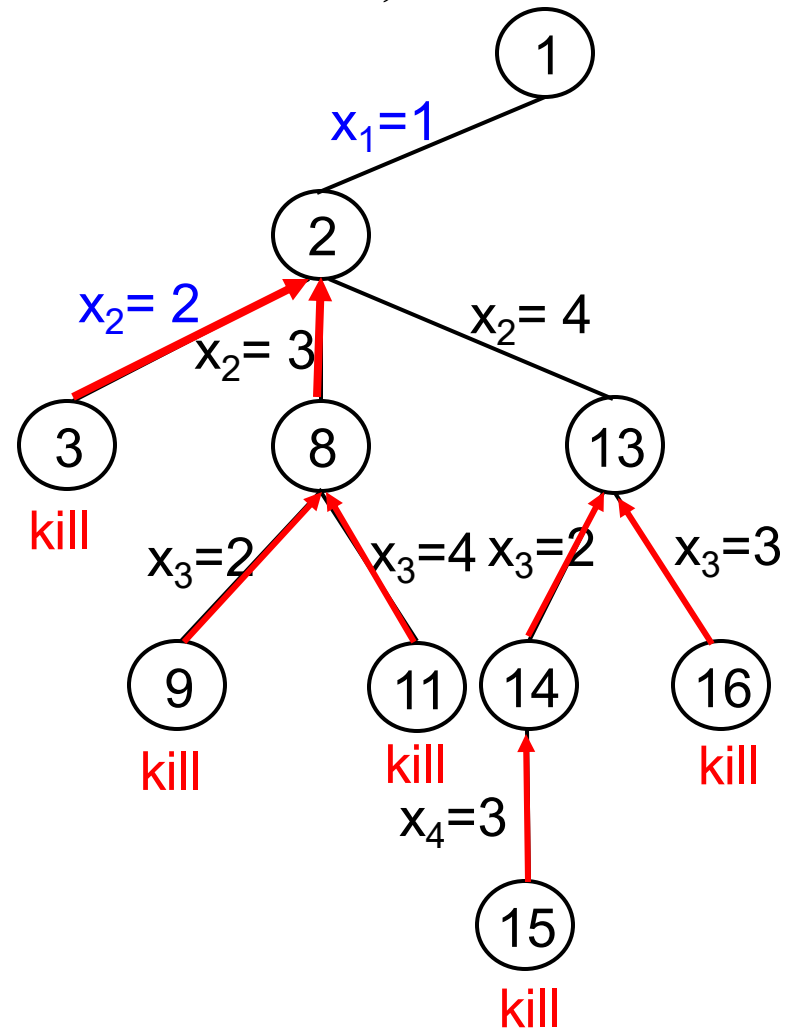
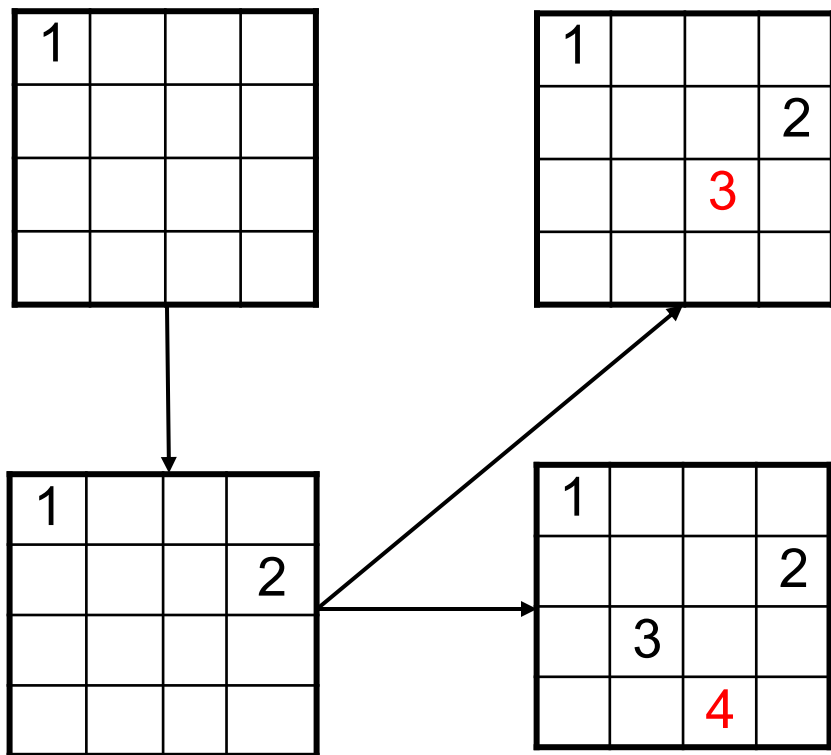
4-皇后问题的状态空间树构造

- 回溯到结点2生成结点8, 路径变为(1, 3), 则结点8成为E-结点, 它生成结点9和结点11都会被杀死, 所以结点8也被杀死, 应回溯



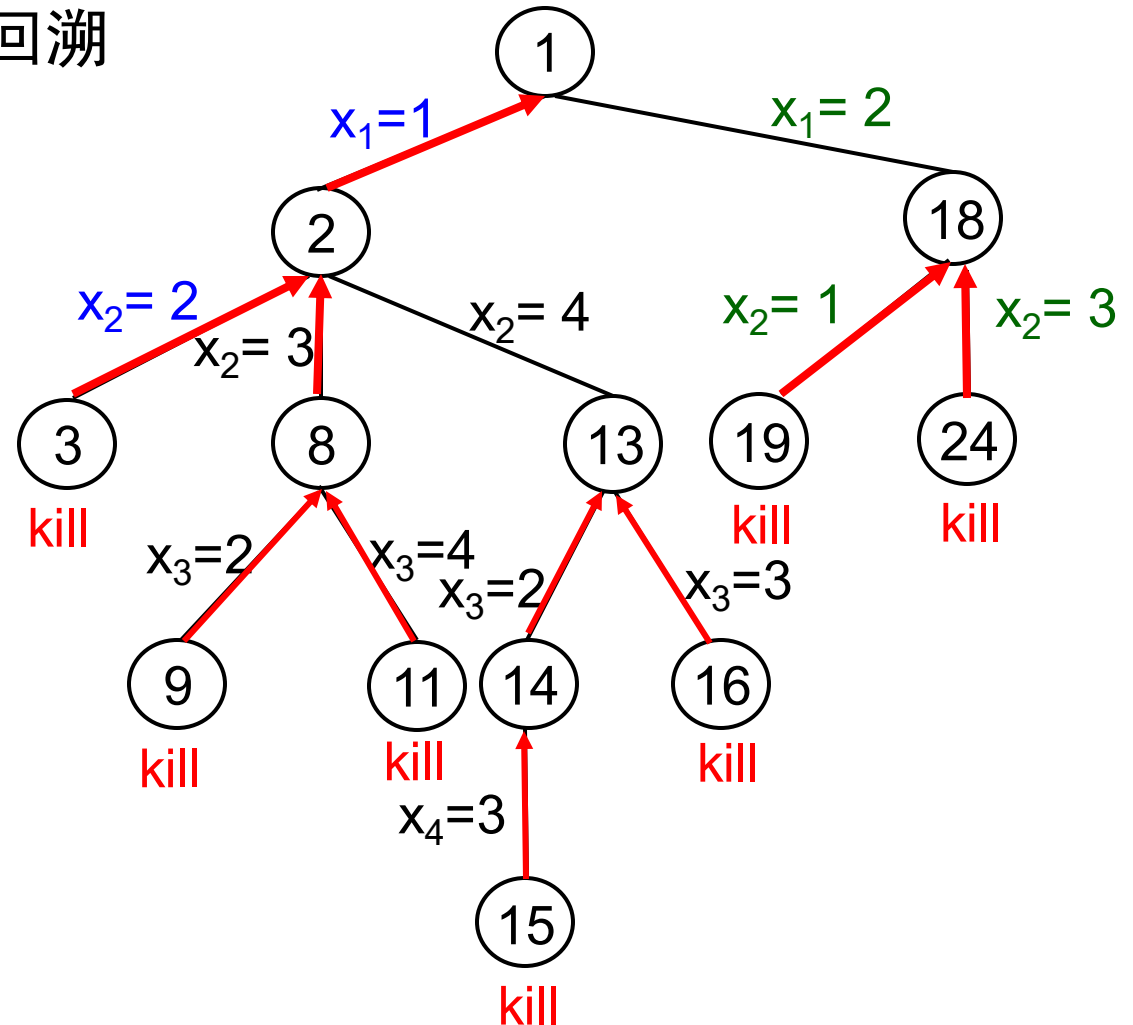
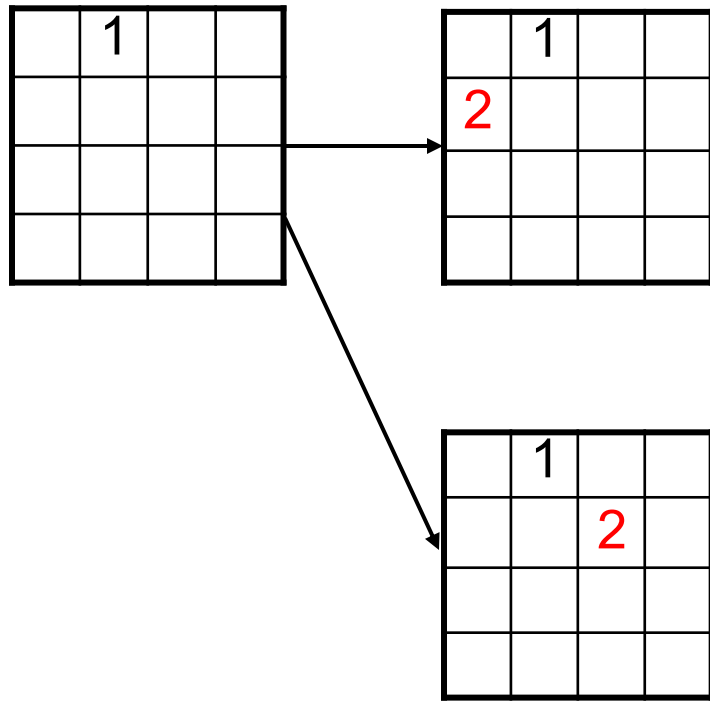
4-皇后问题的状态空间树构造

- 回溯到结点2生成结点13, 路径变为(1, 4), 结点13成为E-结点, 它的孩子不可能导致答案结点, 因此结点13也被杀死, 回溯



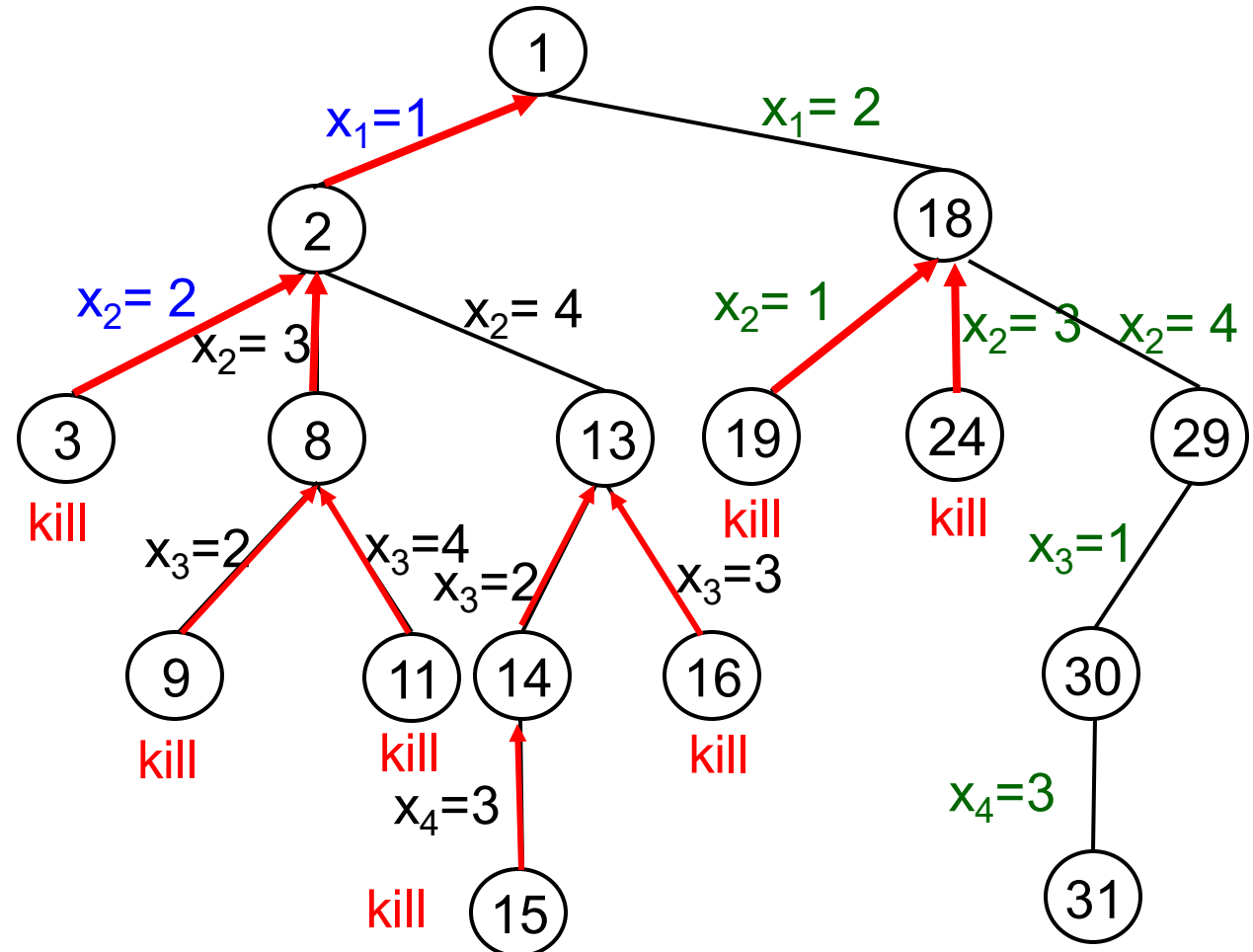
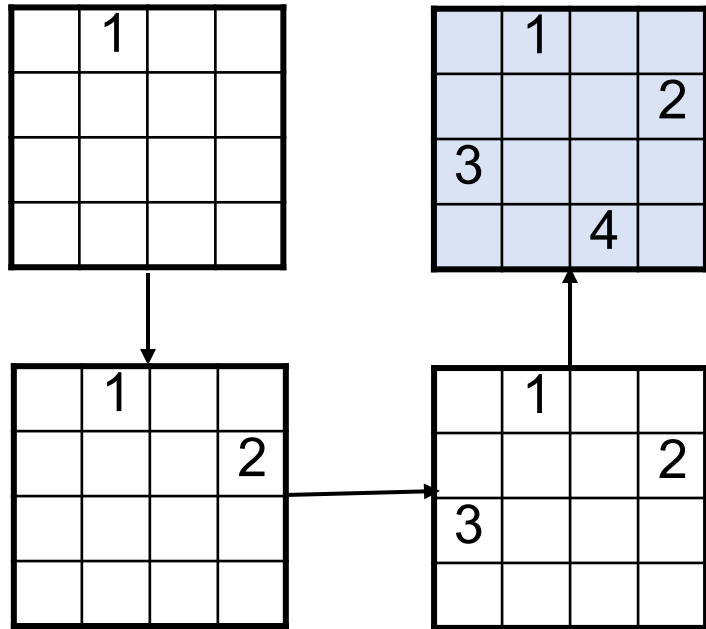
4-皇后问题的状态空间树构造

- 结点2的所有孩子都不能导致答案棋盘格局, 因此结点2也被杀死; 再回溯到结点1生成结点18, 路径变为(2)
- 结点18的孩子结点19、结点24被杀死, 回溯



4-皇后问题的状态空间树构造

- 结点18生成结点29, 结点29成为E-结点, 路径变为(2,4)
- 结点29生成结点30, 路径变为(2,4,1)
- 结点30生成结点31, 路径变为(2,4,1,3), 找到一个4-皇后问题的可行解



- 针对问题定义状态空间树结构：元组、显式约束条件、隐式约束条件
- 根据隐式约束条件设计限界函数
- 检验问题对于限界函数满足多米诺性质
- 以深度优先方式构造状态空间树，在构造过程中使用限界函数避免无效生成
- 直至找到所要求的解或状态空间树中已没有活结点时为止

- 假设要找出**所有**的答案结点
- $(x_1, x_2, \dots, x_{i-1})$ 是状态空间树中由根出发的一条路径，到达结点Y
- $T(x_1, \dots, x_{i-1})$ 是 $(x_1, x_2, \dots, x_{i-1})$ 条件下，元素 x_i 的取值集合，代表结点Y可生成的孩子结点
- 对于限界函数B，如果路径 $(x_1, x_2, \dots, x_{i-1}, x_i)$ 不可能延伸到一个答案结点，则 $B(x_1, x_2, \dots, x_i)$ 取假值，否则取真值

```
procedure BACKTRACK(n)
```

```
  k  $\leftarrow$  1
```

```
  while k > 0 do
```

```
    if T(X(1)...X(k-1))还剩没检验的值X(k) and B(X(1)...X(k))=true then
```

```
      if (X(1)...X(k))是一个解 then
```

```
        print (X(1)...X(k))
```

```
      end
```

```
      k  $\leftarrow$  k+1
```

```
    else
```

```
      k  $\leftarrow$  k-1
```

```
    end
```

```
  end
```

```
end
```

限界函数避免无效搜索

回溯

回溯法的递归算法描述

```
procedure RBACKTRACK(k)
```

```
  for  $X(k) \in T(X(1) \dots X(k-1))$  do
```

```
    if  $(X(1) \dots X(k))$  是一个解 then
```

```
      print  $(X(1) \dots X(k))$ 
```

```
    end
```

```
    if  $B(X(1), \dots, X(k)) = \text{true}$  then
```

```
      RBACKTRACK(k+1)
```

```
    end
```

```
  end
```

```
end
```

进入算法时, 解向量 X 中的前 $k-1$ 个分量
 $X(1) \dots X(k-1)$ 已经被赋值

限界函数避免无效搜索

思考：递归算法中为什么没有回溯？

- 7.1 一般方法
- 7.2 效率估计
- 7.3 n-皇后问题
- 7.4 子集和数问题
- 7.5 图着色问题
- 7.6 小结

- 生成下一个 $X(k)$ 的时间
 - 生成一个结点的时间
- 满足显式约束条件的 $X(k)$ 的数目
 - 子结点的数量
- 限界函数 B 的计算时间
 - 检验结点的时间
- 对于所有的 i , 满足 B 的 $X(k)$ 的数目
 - 通过检验的结点数量

B 能够大大减少生成的结点数，但在计算时间和减少程度上要进行折中

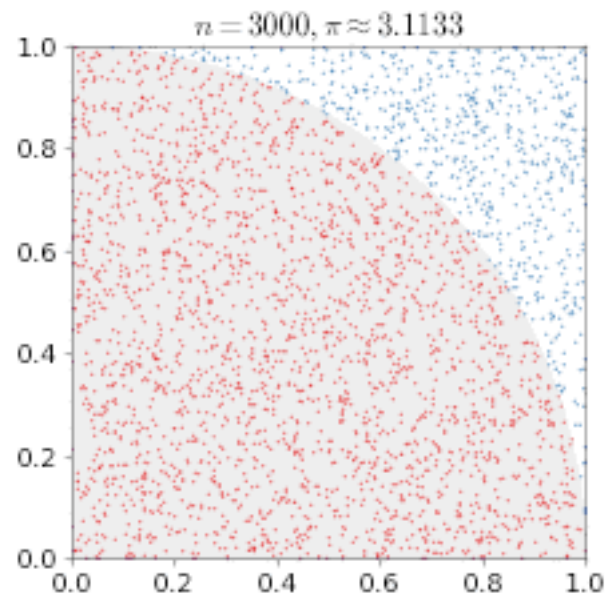
第四个

思考：哪一个因素会导致不同实例产生的结点数不同？

- 如果解空间的结点数是 2^n 或 $n!$ ，易知，回溯算法最坏情况下的时间复杂度为 $O(p(n)2^n)$ 或 $O(q(n)n!)$ ，其中 $p(n)$ 和 $q(n)$ 为 n 的多项式 计算B的时间等
- 由于回溯法对同一问题不同实例的巨大差异，在 n 很大时，对某些实例是十分有效的
 - 在采用回溯法计算某个实例之前，应估算其工作效能
 - 指导限界函数B的设计
- 用回溯算法处理一棵树所要生成的结点数 思考：如何计算？

估计活结点的个数，即
状态空间树结点个数

- 20世纪40年代，在冯·诺伊曼、斯塔尼斯拉夫·乌拉姆和尼古拉斯·梅特罗波利斯于洛斯阿拉莫斯国家实验室为核武器计划工作时，发明了蒙特卡罗方法



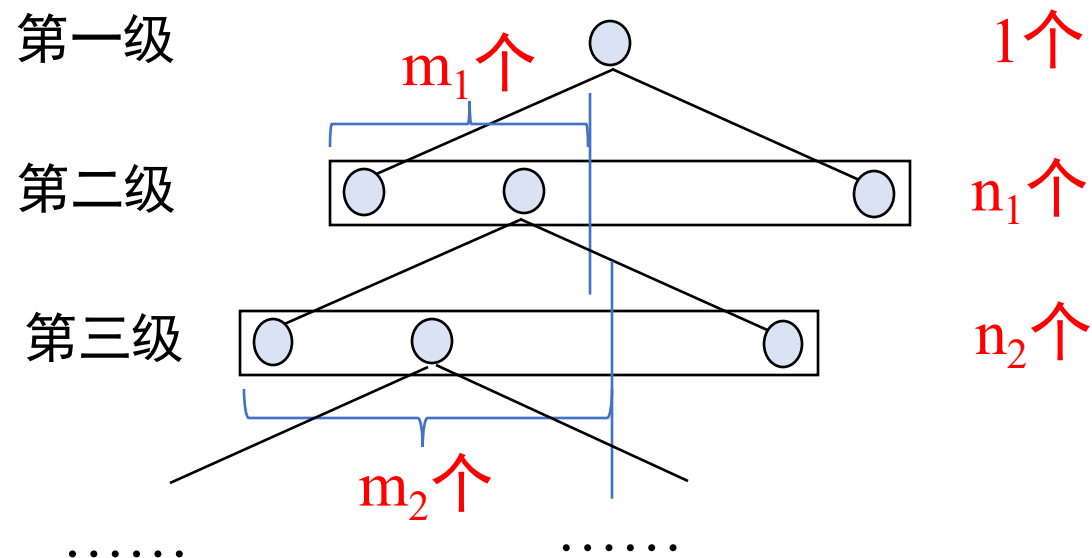
使用蒙特卡罗方法估算 π 值



摩纳哥的蒙特卡洛

- 假定限界函数是**固定**的
 - 在状态空间中生成一条随机路径
 - 设 x 是这条路径上的位于第 i 级的一个结点
 - 设限界函数确定 x 的可用孩子结点的数目为 m_i
 - 从这 m_i 个孩子结点中**随机**选中一个，重复上述过程，直到当前结点是叶结点或者孩子结点都被限界为止

不受限界结点的估计数: $m=1+m_1+m_1*m_2+m_1*m_2*m_3+\dots$



第一级通过B函数检验的结点个数有多少个? 1个

第二级通过B函数检验的结点个数有多少个? m_1 个

从第二级中随机选中一个结点，它的子结点共有 m_2 个满足B函数检验，推断第三级通过B函数检验的结点个数一共有多少个? $\approx m_1 m_2$ 个

Procedure ESTIMATE()

$m \leftarrow 1; r \leftarrow 1; k \leftarrow 1$

loop

$T_k \leftarrow \{X(k) : X(k) \in T(X(1), \dots, X(k-1)) \text{ and } B_k(X(1), \dots, X(k))\}$

if SIZE(T_k)=0 then exit end

$r \leftarrow r * \text{SIZE}(T_k)$

第k级的结点数

$m \leftarrow m + r$

前k级的结点总数

$X(k) \leftarrow \text{CHOOSE}(T_k)$

$k \leftarrow k + 1$

从 T_k 中随机地挑选一个元素

end

return m

end

运行多次ESTIMATE，取平均，使估计值更加准确

- 优点：
 - 对于找到**所有**答案结点的情况非常有用
 - 限界函数固定不变，计算方便，对状态空间树中同一级结点都适用
- 缺点：
 - 只求**一个**解时，生成的结点数远小于 m
 - 随着检索的进行，限界函数应该更强，使得 m 的值更小

- 7.1 一般方法
- 7.2 效率估计
- 7.3 n-皇后问题
- 7.4 子集和数问题
- 7.5 图着色问题
- 7.6 小结

- 输入：棋盘大小 n
- 输出： n 个皇后的位置 $x_1, x_2, \dots, x_n$ ， x_i 表示第 i 个皇后放在 (i, x_i) 处，使得

任意两个 x_i 都不相同, 且没有两个皇后可以在同一条斜角线上

- 输入：棋盘大小 n
- 输出： n 个皇后的位置 $x_1, x_2, \dots, x_n$ ， x_i 表示第 i 个皇后放在 (i, x_i) 处，使得

任意两个 x_i 都不相同, 且没有两个皇后可以在同一条斜角线上

- 显式约束: $S_i = \{1, 2, \dots, n\}$, $1 \leq i \leq n$
- 隐式约束: $x_1 \neq x_2 \neq \dots \neq x_n$ ，任意两个 x_i 不能同在同一条斜角线上
- 解空间: $\{1, 2, \dots, n\}^n$  大小为 n^n
 - 由 $x_1 \neq x_2 \neq \dots \neq x_n$ 进一步限制  大小为 $n!$

任意两个 x_i 都不相同, 且没有两个皇后可以在同一条斜角线上

- 同一条斜角线上的每个元素
 - 由左到右具有相同的“行-列”值;
 - 由右到左具有相同的“行+列”值。

a_{11}	a_{12}	a_{13}	a_{14}	a_{15}	a_{16}	a_{17}	a_{18}
a_{21}	a_{22}	a_{23}	a_{24}	a_{25}	a_{26}	a_{27}	a_{28}
a_{31}	a_{32}	a_{33}	a_{34}	a_{35}	a_{36}	a_{37}	a_{38}
a_{41}	a_{42}	a_{43}	a_{44}	a_{45}	a_{46}	a_{47}	a_{48}
a_{51}	a_{52}	a_{53}	a_{54}	a_{55}	a_{56}	a_{57}	a_{58}
a_{61}	a_{62}	a_{63}	a_{64}	a_{65}	a_{66}	a_{67}	a_{68}
a_{71}	a_{72}	a_{73}	a_{74}	a_{75}	a_{76}	a_{77}	a_{78}
a_{81}	a_{82}	a_{83}	a_{84}	a_{85}	a_{86}	a_{87}	a_{88}

任意两个 x_i 都不相同, 且没有两个皇后可以在同一条斜角线上

设有两个皇后位于 $(i, X(i))$ 和 $(k, X(k))$ 位置上

$$\left. \begin{array}{l} i - X(i) = k - X(k) \\ i + X(i) = k + X(k) \end{array} \right\} \longrightarrow |X(i) - X(k)| = |i - k|$$

// 前 $k-1$ 行的皇后已经放置, 现在确定第 k 行皇后欲放在 $X(k)$ 列上, 是否可行?

PLACE(k)

令 $X(k)$ 与 $X(i)$ 逐个比较, $i=1..k-1$

若存在 $X(k)=X(i)$ 或者 $|X(i)-X(k)|=|i-k|$

则返回false

否则返回true

//若一个皇后能放在第k行和第X(k)列, 则返回true, 否则返回false

//X是全程数组, 进入此过程时已置入了k个值, ABS是绝对值函数

```
procedure PLACE(k)
  i ← 1
  while ( i < k ) do
    if  $X(i) = X(k)$  or  $ABS(X(i) - X(k)) = ABS(i - k)$  then
      return false
    end
    i ← i + 1
  end
  return true
end
```

```
procedure BACKTRACK(n)
```

```
  k ← 1
```

```
  while k>0 do
```

```
    if T(X(1)...X(k-1))还剩没检验的值X(k) and B(X(1)...X(k))=true then
```

```
      if (X(1)...X(k))是一个解 then
```

```
        print (X(1)...X(k))
```

```
      end
```

```
      k ← k+1
```

```
    else
```

```
      k ← k-1
```

```
    end
```

```
  end
```

```
end
```

限界函数避免无效搜索

回溯

n-皇后问题的回溯算法

```
procedure NQUEENS(n)
```

```
  X(1)  $\leftarrow$  0 ; k  $\leftarrow$  1
```

```
  while k > 0 do
```

```
    X(k)  $\leftarrow$  X(k)+1
```

思考：能否省略？

```
    while X(k)  $\leq$  n and not PLACE(k) do
```

```
      X(k)  $\leftarrow$  X(k)+1 //当前列X(k)不能放皇后k时，放到下一列
```

```
    end
```

```
    if X(k)  $\leq$  n then
```

```
      if k = n then
```

```
        print (X)
```

```
      else
```

```
        k  $\leftarrow$  k+1; X(k)  $\leftarrow$  0 //准备求解下一个皇后
```

思考：能否省略？

```
      end
```

```
    else
```

```
      k  $\leftarrow$  k-1; //没有合适的位置, 回溯
```

```
    end
```

```
  end
```

```
end
```

- 在8-皇后问题中, $(x_1, x_2, \dots, x_8)$ 表示一个解, x_i 表示第*i*个皇后放在第*i*行的列数。
 - 显式: $S_i = \{1, 2, 3, 4, 5, 6, 7, 8\}, 1 \leq i \leq 8$
 - 隐式: 没有两个 x_i 可以相同,
且没有两个皇后可以在同一条斜角线上
- 蛮力法: 8^8
 - 状态空间树结点个数: $1 + 8 + 8^2 + \dots + 8^8$
- 没有两个 x_i 可以相同: $8!$
 - 状态空间树结点个数: $1 + 8 + 8 * 7 + 8 * 7 * 6 + \dots + 8 * 7 * 6 * 5 * 4 * 3 * 2 * 1 = 69281$
- 限界函数实现隐式约束条件?

使用蒙特卡罗方法估计

	1						
			2				
3							
		4					
			5				

8 1+
5 8+
4 8*5+
3 8*5*4+
2 8*5*4*3+
8*5*4*3*2
=1649

1							
							2
					3		
		4					
						5	
	6						
			7				

8
6
4
2
1
1
1
=1401

多次实验后取平均值1625

不受限结点的估计数大约是8-皇后状态空间树的结点总数的

$$1625/69281=2.34\%$$

- 7.1 一般方法
- 7.2 效率估计
- 7.3 n-皇后问题
- 7.4 子集和数问题
- 7.5 图着色问题
- 7.6 小结

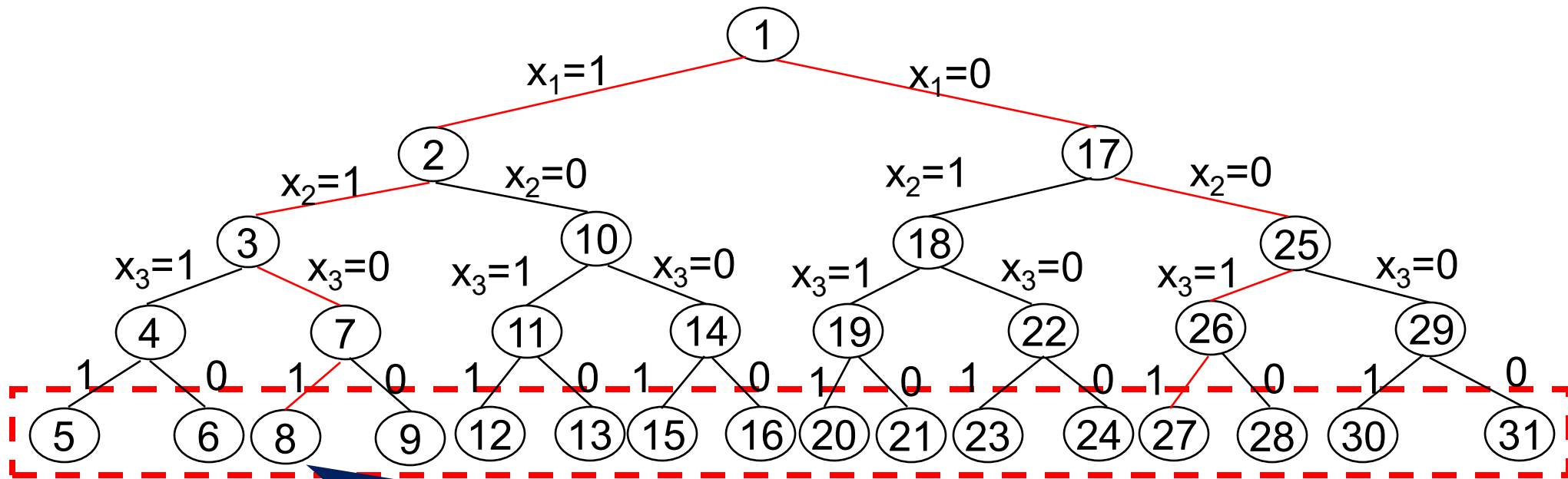
- 输入： $n+1$ 个正数 $w_i (1 \leq i \leq n)$ 和 M
- 输出： 找出 w_i 的和数是 M 的 **所有** 子集

- 针对问题定义状态空间树结构：元组、显式约束条件、隐式约束条件

- n 元组表达 $(x_1, x_2, \dots, x_n)$, x_i 表示是否选择 w_i , 如果选择 w_i , 则 $x_i=1$; 否则 $x_i=0$ 。
 - 显式约束: $x_i \in \{0, 1\}, 1 \leq i \leq n$;
 - 隐式约束: $\sum w_i x_i = M, 1 \leq i \leq n$
- 例: $n=4, (w_1, w_2, w_3, w_4) = (11, 13, 24, 7), M=31$
 - 可行解1: 11, 13, 7
 - 可行解2: 24, 7
- 固定长元组: 4-元组:
 - 可行解1: (1, 1, 0, 1)
 - 可行解2: (0, 0, 1, 1)
- 解空间共计 $2^n = 2^4 = 16$ 个元组

固定长元组的状态空间树

- 问题状态：全部结点31个
- 解状态：叶结点16个
- 答案状态：当前实例2个



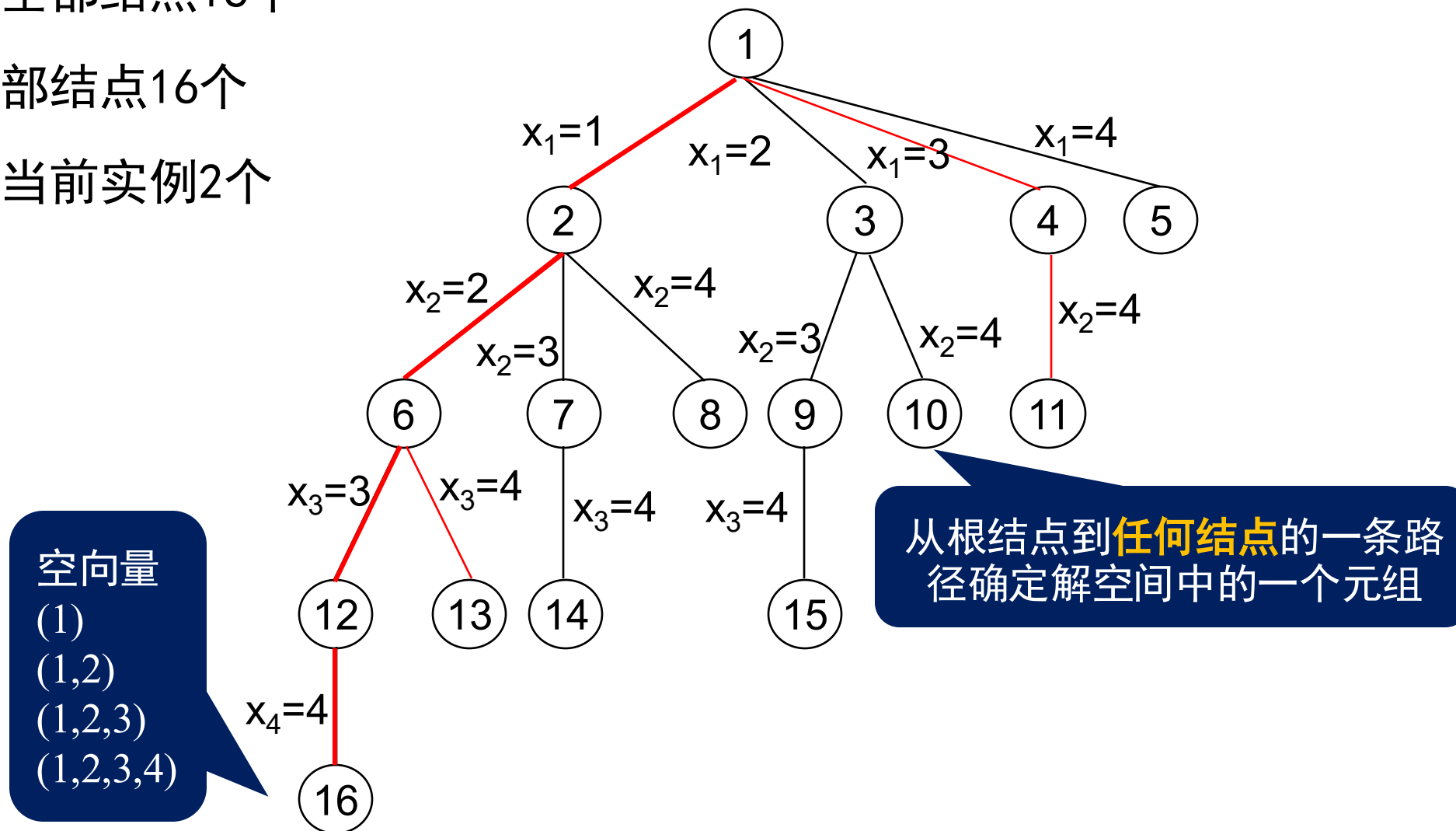
从根结点到叶结点的一条路径确定解空间中的一个元组

- K-元组表达, $(x_1, \dots, x_k)$, $1 \leq k \leq n$,
 - 显式约束: $x_i \in \{j \mid j \text{ 是 } w_j \text{ 的下标值}, 1 \leq j \leq n\}$, $1 \leq i \leq k$, 如果选择 w_i , 则 w_i 的下标在 k -元组中。不同的解 **k值不同**
 - 隐式约束: 没有两个 x_i 是相同的, 且相应的 w_i 的和等于 M , $x_i \leq x_{i+1}$, $1 \leq i < k$
- 例: $n=4$, $(w_1, w_2, w_3, w_4) = (11, 13, 24, 7)$, $M=31$
 - 可行解1: 11, 13, 7
 - 可行解2: 24, 7
- k -元组:
 - 可行解1: (1, 2, 4)
 - 可行解2: (3, 4)
- 解空间共计 $2^n = 2^4 = 16$ 个元组

避免重复情况,
如(1,2,4)和(1,4,2)

可变长元组的状态空间树

- 问题状态：全部结点16个
- 解状态：全部结点16个
- 答案状态：当前实例2个



- 针对问题定义状态空间树结构：元组、显式约束条件、隐式约束条件
- 根据隐式约束条件设计限界函数
- 检验问题对于限界函数满足多米诺性质

隐式约束： w_i 的和数是M

- 使用固定长元组： x_i 表示是否选择 w_i
- 由 $x_1, \dots, x_k$ 选择的数之和小于等于M
 - $\sum_{i=1}^k x_i \cdot w_i \leq M$
- 如果一开始 w_i 按非降次序排列，那么
 - $\sum_{i=1}^k x_i \cdot w_i + w_{k+1} \leq M$
- 剩余的数之和足够达到M
 - $\sum_{i=1}^k x_i \cdot w_i + \sum_{i=k+1}^n w_i \geq M$

满足多米诺性质！

综上，限界函数 $B(x_1, \dots, x_k) = \text{true}$ ，当且仅当：

$$\sum_{i=1}^k x_i \cdot w_i + w_{k+1} \leq M \quad \text{且} \quad \sum_{i=1}^k x_i \cdot w_i + \sum_{i=k+1}^n w_i \geq M$$

- 针对问题定义状态空间树结构：元组、显式约束条件、隐式约束条件
- 根据隐式约束条件设计限界函数
- 检验问题对于限界函数满足多米诺性质
- 以深度优先方式构造状态空间树，在构造过程中使用限界函数避免无效生成
- 直至找到所要求的解或状态空间树中已没有活结点时为止


```
procedure RBACKTRACK(k)
```

```
  for  $X(k) \in T(X(1) \dots X(k-1))$  do
```

```
    if  $(X(1) \dots X(k))$  是一个解 then
```

```
      print  $(X(1) \dots X(k))$ 
```

```
    end
```

```
    if  $B(X(1), \dots, X(k)) = \text{true}$  then
```

```
      RBACKTRACK(k+1)
```

```
    end
```

```
  end
```

```
end
```

进入算法时, 解向量 X 中的前 $k-1$ 个分量
 $X(1) \dots X(k-1)$ 已经被赋值

限界函数避免无效搜索

子集和数的递归回溯算法

//找出W(1:n)中和数为M的所有子集
//进入此过程时X(1),..X(k-1)的值已确定
//W(j)按非降次序排列

Procedure SUMOFSUB(k)

X(k)←1

if $\sum_{i=1}^k X(i) \cdot W(i) = M$ then
 print(X)

end

if $\sum_{i=1}^k X(i) \cdot W(i) + W(k+1) \leq M$ then
 SUMOFSUB(k+1)

end

X(k)←0

if $\sum_{i=1}^k X(i) \cdot W(i) + W(k+1) \leq M$ and $\sum_{i=1}^k X(i) \cdot W(i) + \sum_{i=k+1}^n W(i) \geq M$ then
 SUMOFSUB(k+1)

end

end

限界函数 $B(x_1, \dots, x_k) = \text{true}$, 当且仅当:

$$\sum_{i=1}^k x_i \cdot w_i + w_{k+1} \leq M \quad \text{且} \quad \sum_{i=1}^k x_i \cdot w_i + \sum_{i=k+1}^n w_i \geq M$$

思考: 为何不判断 $\sum_{i=1}^k X(i) \cdot W(i) + \sum_{i=k+1}^n W(i) \geq M$?

思考: 重复计算, 是否可以优化?

//找出W(1:n)中和数为M的所有子集

//进入此过程时X(1),..X(k-1)的值已确定

//W(j)按非降次序排列

Procedure SUMOFSUB(**s**, k, **r**)

 X(k)←1

 if s+W(k) = M then

 print(X)

 end

 if s+W(k)+W(k+1) ≤ M then

 SUBOFSUB(s+W(k), k+1, r-W(k))

 end

 X(k)←0

 if s+W(k+1) ≤ M and s+r-W(k) ≥ M then

 SUMOFSUB(s, k+1, r-W(k))

 end

end

限界函数B($x_1, \dots, x_k$)=true, 当且仅当:

$$\sum_{i=1}^k x_i \cdot w_i + w_{k+1} \leq M \quad \text{且} \quad \sum_{i=1}^k x_i \cdot w_i + \sum_{i=k+1}^n w_i \geq M$$

$$\mathbf{s} = \sum_{i=1}^{k-1} \mathbf{X}(i) \cdot \mathbf{W}(i), \quad \mathbf{r} = \sum_{i=k}^n \mathbf{W}(i)$$

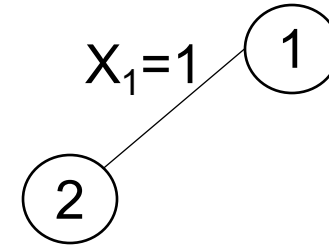
• $M=30, W=(5,10,12,13,15,18)$

①

结点编号	s	r	W(i+1)	B值

- $M=30, W=(5,10,12,13,15,18)$

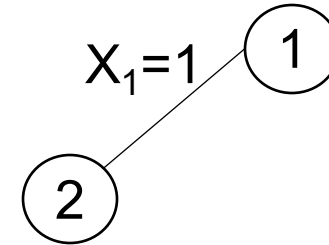
结点编号	s	r	$W(i+1)$	B值
2	5	68	10	



$$s+W(i+1) \leq M \text{ 且 } s+r \geq M$$

- $M=30, W=(5,10,12,13,15,18)$

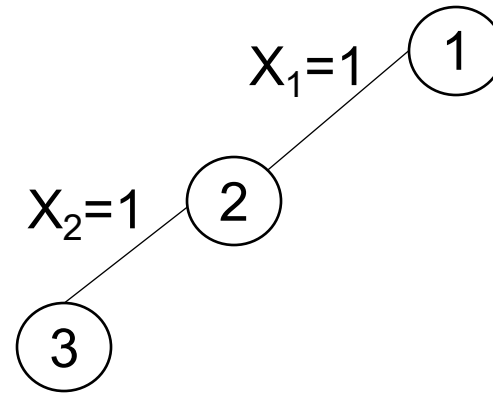
结点编号	s	r	$W(i+1)$	B值
2	5	68	10	T



$$s+W(i+1) \leq M \text{ 且 } s+r \geq M$$

- $M=30, W=(5,10,12,13,15,18)$

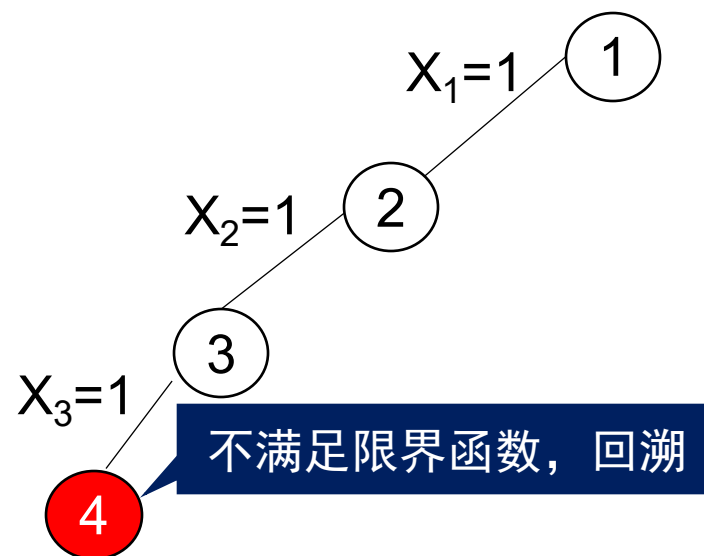
结点编号	s	r	$W(i+1)$	B值
2	5	68	10	T
3	15	58	12	T



$$s+W(i+1) \leq M \text{ 且 } s+r \geq M$$

- $M=30, W=(5,10,12,13,15,18)$

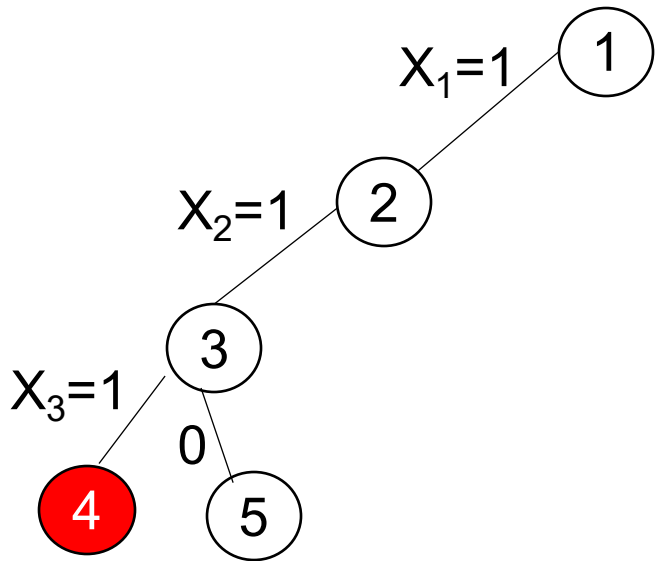
结点编号	s	r	W(i+1)	B值
2	5	68	10	T
3	15	58	12	T
4	27	46	13	F



$$s+W(i+1) \leq M \text{ 且 } s+r \geq M$$

- $M=30, W=(5,10,12,13,15,18)$

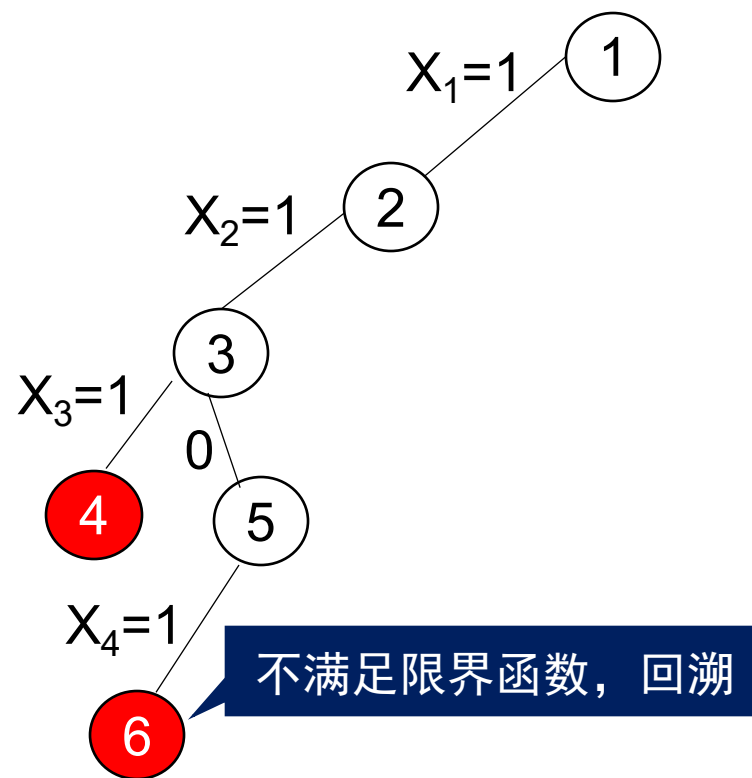
结点编号	s	r	W(i+1)	B值
2	5	68	10	T
3	15	58	12	T
4	27	46	13	F
5	15	46	13	T



$s+W(i+1) \leq M \text{ 且 } s+r \geq M$

- $M=30, W=(5,10,12,13,15,18)$

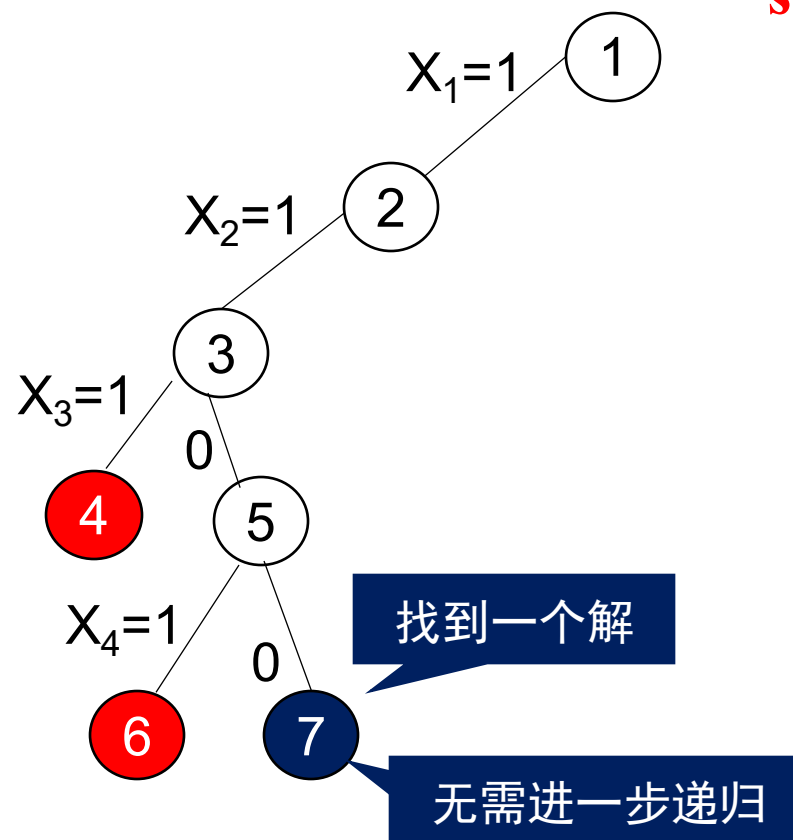
结点编号	s	r	$W(i+1)$	B值
2	5	68	10	T
3	15	58	12	T
4	27	46	13	F
5	15	46	13	T
6	28	33	15	F



$$s+W(i+1) \leq M \text{ 且 } s+r \geq M$$

- $M=30, W=(5,10,12,13,15,18)$

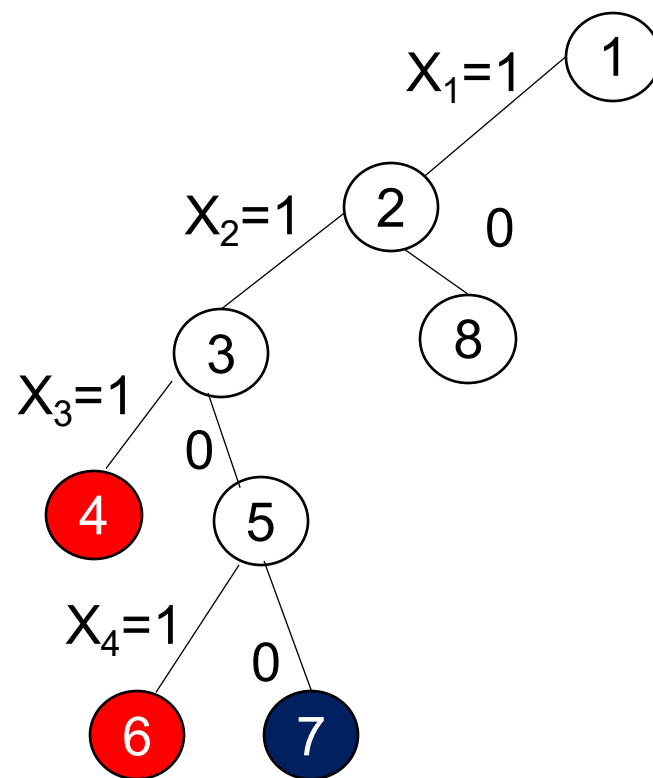
结点编号	s	r	$W(i+1)$	B值
2	5	68	10	T
3	15	58	12	T
4	27	46	13	F
5	15	46	13	T
6	28	33	15	F
7	15	33	15	T



$$s+W(i+1) \leq M \text{ 且 } s+r \geq M$$

- $M=30, W=(5,10,12,13,15,18)$

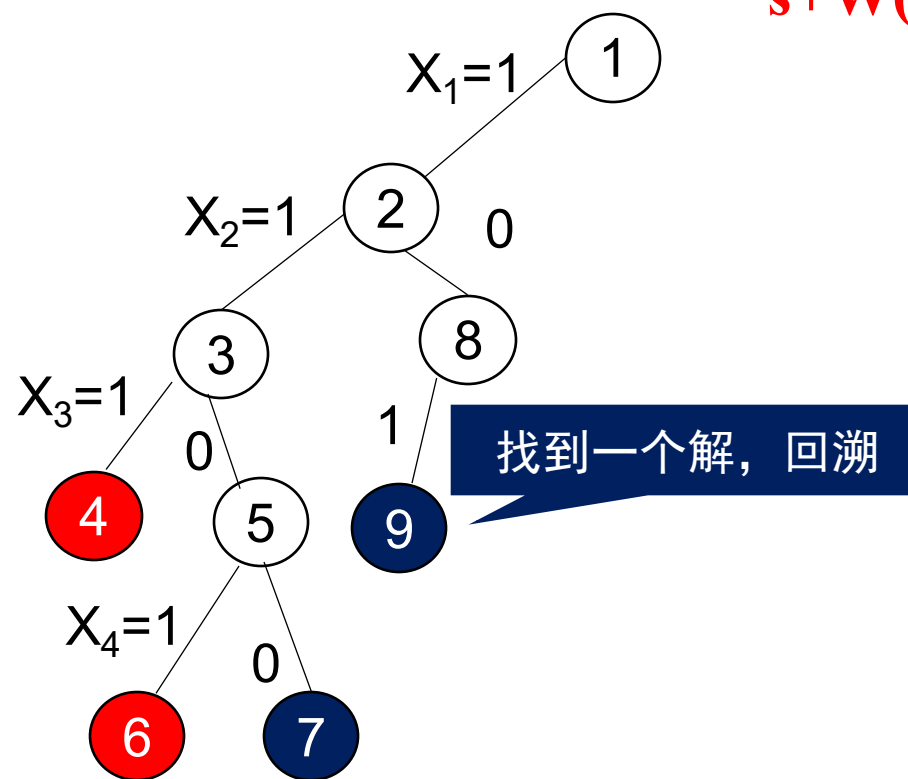
结点编号	s	r	W(i+1)	B值
2	5	68	10	T
3	15	58	12	T
4	27	46	13	F
5	15	46	13	T
6	28	33	15	F
7	15	33	15	T
8	5	58	12	T



$$s+W(i+1) \leq M \text{ 且 } s+r \geq M$$

- $M=30, W=(5,10,12,13,15,18)$

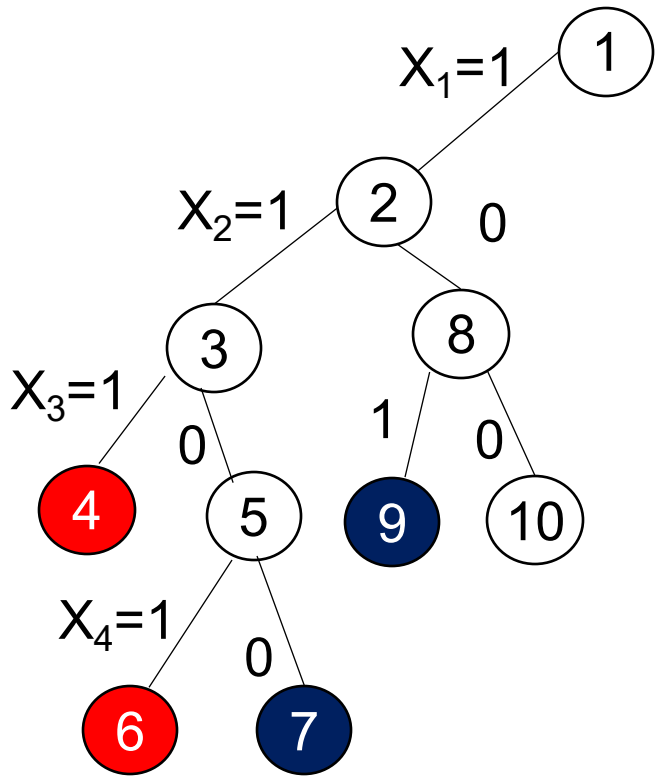
结点编号	s	r	$W(i+1)$	B值
2	5	68	10	T
3	15	58	12	T
4	27	46	13	F
5	15	46	13	T
6	28	33	15	F
7	15	33	15	T
8	5	58	12	T
9	17	46	13	T



$$s+W(i+1) \leq M \text{ 且 } s+r \geq M$$

- $M=30, W=(5,10,12,13,15,18)$

结点编号	s	r	W(i+1)	B值
2	5	68	10	T
3	15	58	12	T
4	27	46	13	F
5	15	46	13	T
6	28	33	15	F
7	15	33	15	T
8	5	58	12	T
9	17	46	13	T
10	5	46	13	T

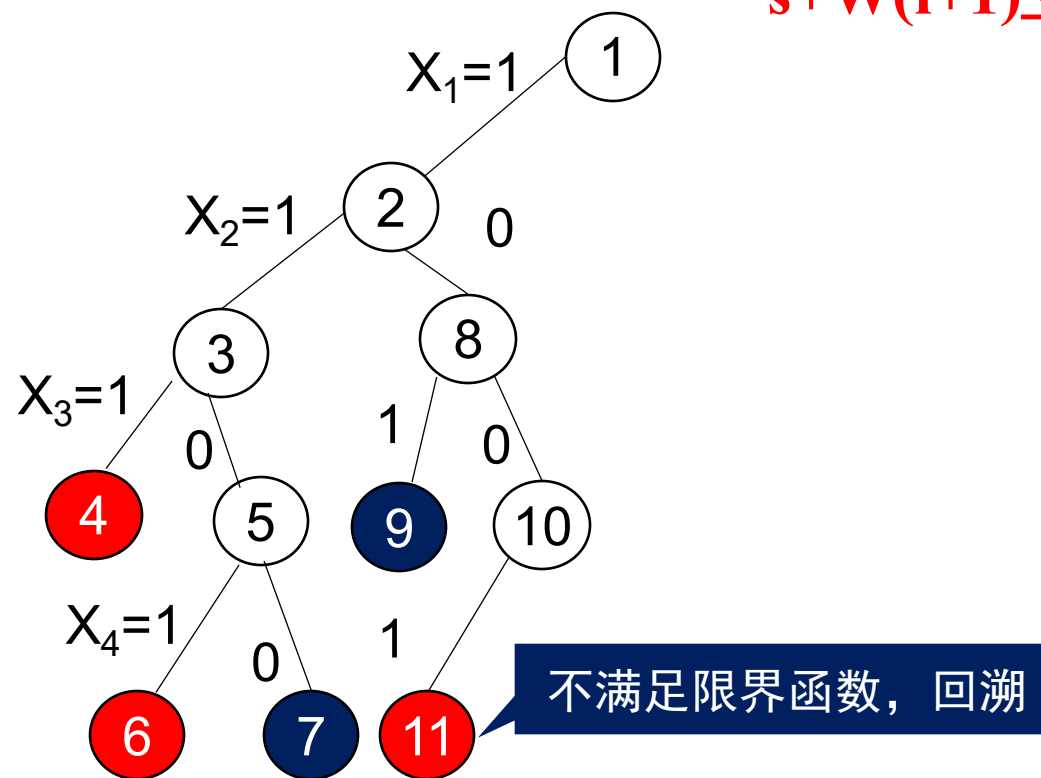


$s+W(i+1) \leq M \text{ 且 } s+r \geq M$

- $M=30, W=(5,10,12,13,15,18)$

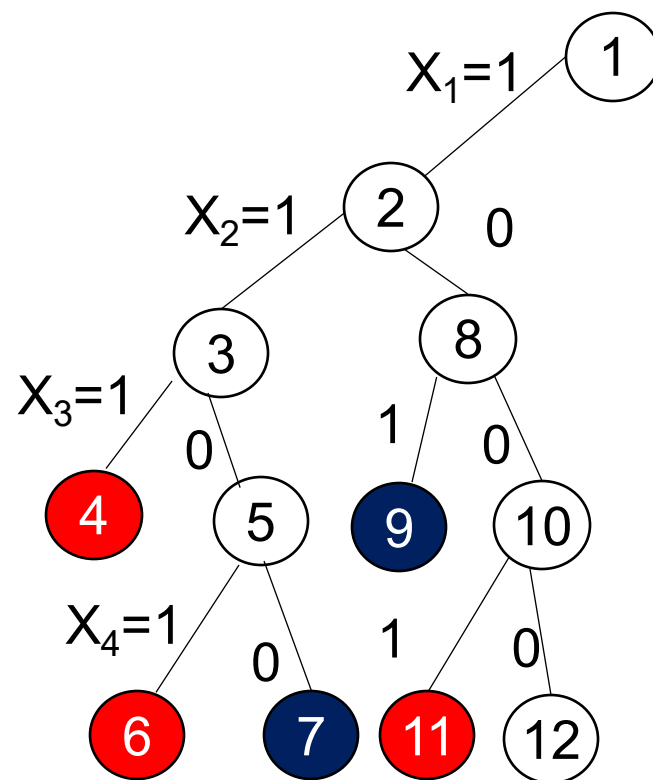
结点编号	s	r	$W(i+1)$	B值
2	5	68	10	T
3	15	58	12	T
4	27	46	13	F
5	15	46	13	T
6	28	33	15	F
7	15	33	15	T
8	5	58	12	T
9	17	46	13	T
10	5	46	13	T
11	18	33	15	F

$$s+W(i+1) \leq M \text{ 且 } s+r \geq M$$



- $M=30, W=(5,10,12,13,15,18)$

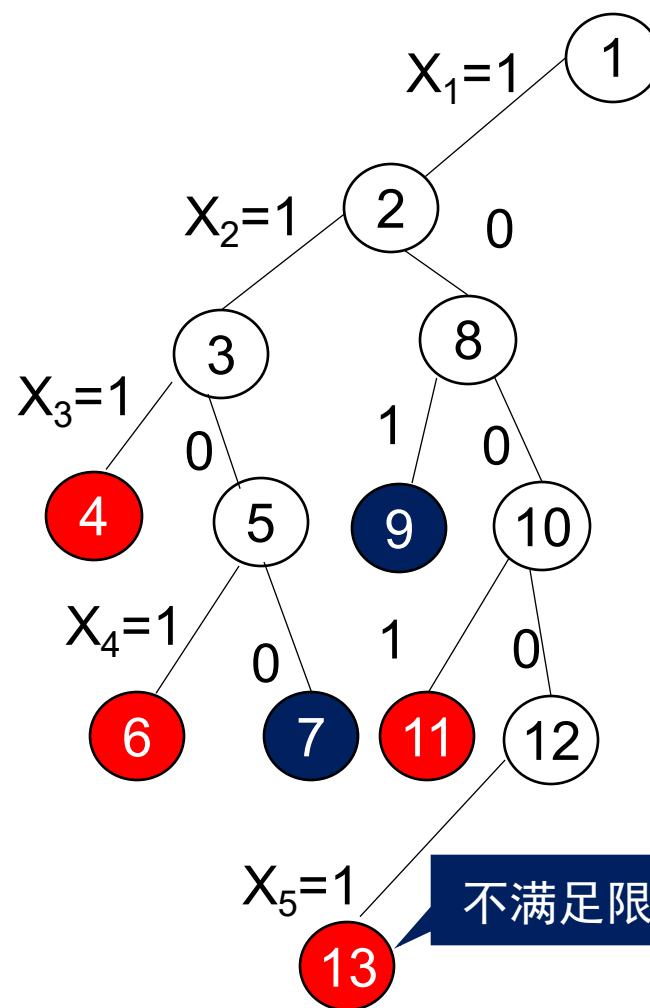
结点编号	s	r	W(i+1)	B值
2	5	68	10	T
3	15	58	12	T
4	27	46	13	F
5	15	46	13	T
6	28	33	15	F
7	15	33	15	T
8	5	58	12	T
9	17	46	13	T
10	5	46	13	T
11	18	33	15	F
12	5	33	15	T



$$s+W(i+1) \leq M \text{ 且 } s+r \geq M$$

- $M=30, W=(5,10,12,13,15,18)$

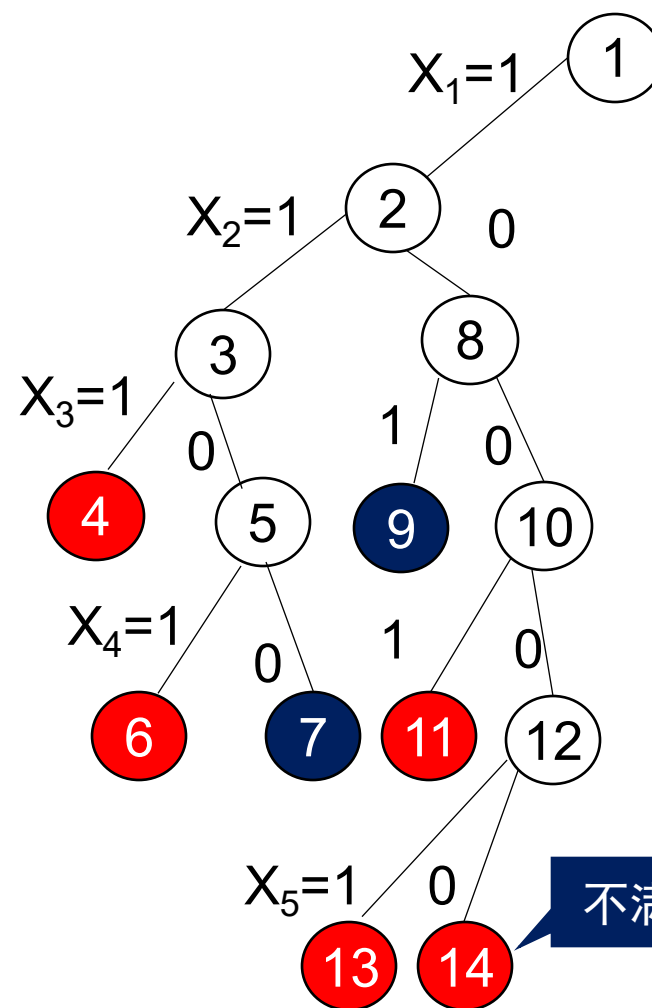
结点编号	s	r	W(i+1)	B值
2	5	68	10	T
3	15	58	12	T
4	27	46	13	F
5	15	46	13	T
6	28	33	15	F
7	15	33	15	T
8	5	58	12	T
9	17	46	13	T
10	5	46	13	T
11	18	33	15	F
12	5	33	15	T
13	20	18	18	F



$$s+W(i+1) \leq M \text{ 且 } s+r \geq M$$

- $M=30, W=(5,10,12,13,15,18)$

结点编号	s	r	W(i+1)	B值
2	5	68	10	T
3	15	58	12	T
4	27	46	13	F
5	15	46	13	T
6	28	33	15	F
7	15	33	15	T
8	5	58	12	T
9	17	46	13	T
10	5	46	13	T
11	18	33	15	F
12	5	33	15	T
13	20	18	18	F
14	5	18	18	F

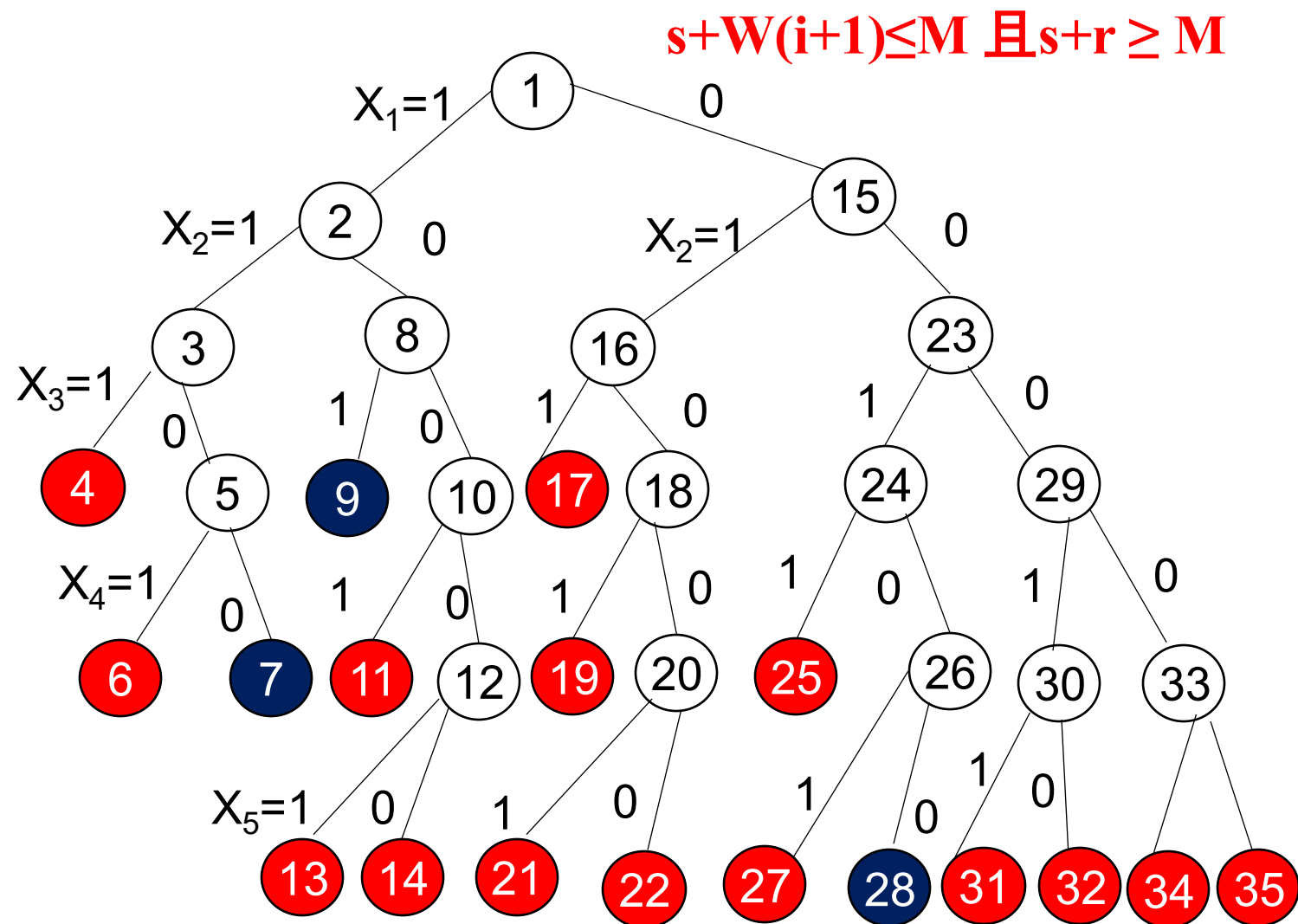


$$s+W(i+1) \leq M \text{ 且 } s+r \geq M$$

不满足限界函数，回溯

- $M=30, W=(5,10,12,13,15,18)$

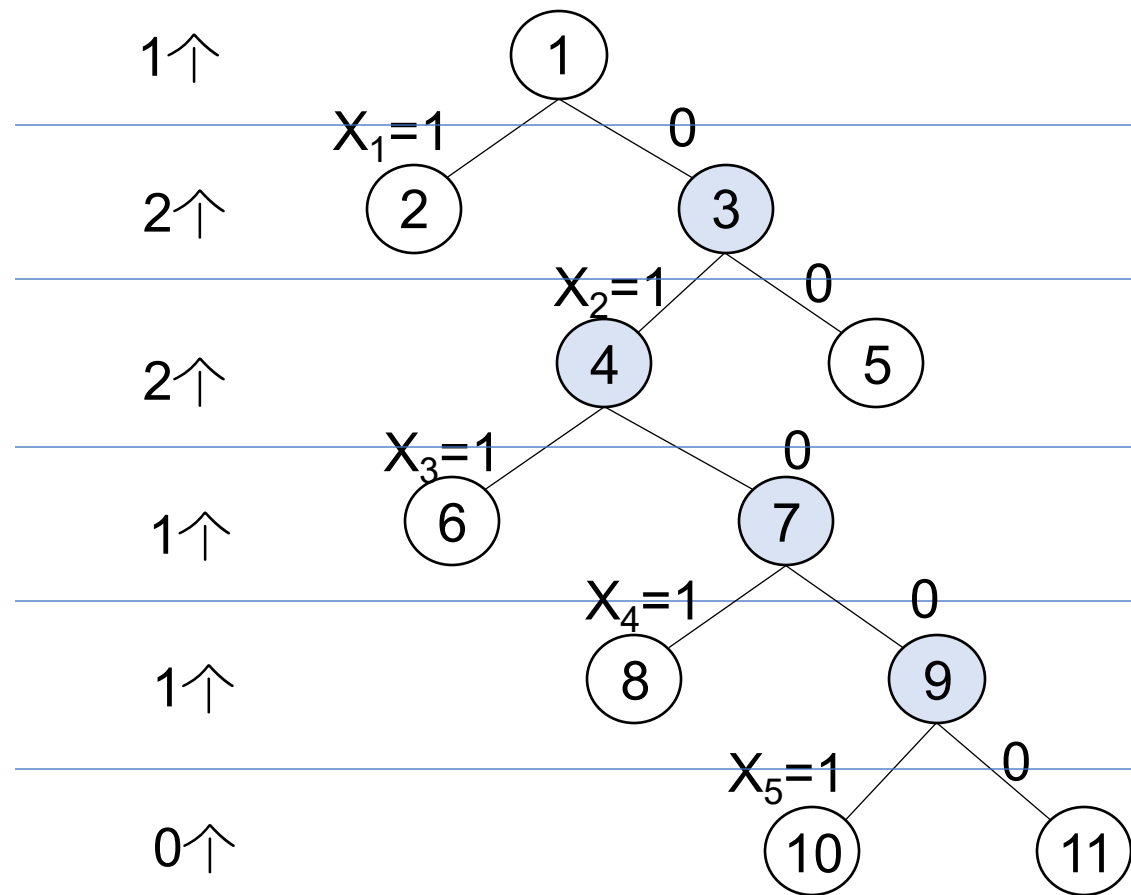
结点编号	s	r	W(i+1)	B值
2	5	68	10	T
3	15	58	12	T
4	27	46	13	F
5	15	46	13	T
6	28	33	15	F
7	15	33	15	T
8	5	58	12	T
9	17	46	13	T
10	5	46	13	T
11	18	33	15	F
12	5	33	15	T
13	20	18	18	F
14	5	18	18	F



使用限界函数后，共生成35个结点 $< 2^6=64$

- $M=30, W=(5,10,12,13,15,18)$

结点编号	s	r	W(i+1)	B值
2	5	68	10	T
3	0	68	10	T
4	10	58	12	T
5	0	58	12	T
6	22	46	13	F
7	10	46	13	T
8	23	33	15	F
9	10	33	15	T
10	25	18	18	F
11	10	18	18	F



不受限界结点的估计数: $m=1+2+2*2+2*2*1+2*2*1*1+0=15$

- 7.1 一般方法
- 7.2 效率估计
- 7.3 n-皇后问题
- 7.4 子集和数问题
- 7.5 图着色问题
- 7.6 小结

- 回溯法适用的问题
 - 多阶段决策问题/组合问题
 - 满足多米诺性质

- 回溯法的设计思想概述
 - 确定解向量：
 - 固定 n -元组
 - 可变 k -元组
 - 分解约束条件：
 - 显示约束
 - 隐式约束
 - 确定状态空间树
 - 设计限界函数 B
 - 深度优先方式搜索树

- 状态空间树的分类
 - 集合树：问题的解是对已知集合元素的取舍
 - 如子集和数问题，0/1背包问题
 - 排列树：问题的解是对已知集合元素的排列
 - 如n-皇后问题，图着色判定问题
- 回溯法的效率问题
 - 状态空间树的大小：决定最坏情况
 - 限界函数B的剪枝能力：决定状态空间树大小
 - 求问题全部的解时，可以用蒙特卡洛方法估计算法效率

- 回溯法的改进
 - 根据树的分支情况设计优先策略，如优先搜索结点少/解多的分支
 - 利用搜索树的对称性对子树进行剪裁
- n-皇后问题
- 子集和数问题

根据目标函数设计优先策略——分支限界法！